Web.Lib

FREE EDITION

100 AI Prompts Every Programmer Should Have

The Copy-Paste Prompt Library for Front-End, Back-End, Debugging, Design & Career Growth

Build faster. Debug smarter. Ship like a senior engineer.

Next.js · React · TypeScript · Tailwind · Node.js · Prisma

Compatible with ChatGPT · Claude · Gemini

VERSION 1.0

GETTING STARTED

How to Use This Book

Every prompt in this book is a finished, professional-grade instruction you can copy directly into ChatGPT, Claude, or Gemini. You don't need to be a "prompt engineer" — the engineering has already been done for you.

Each prompt follows the same simple structure:

- **What is this prompt for?** — when to reach for it.
- **AI Prompt** — the complete, copy-paste-ready text. Replace anything in `[SQUARE BRACKETS]` with your own details.
- **Example Result** — what the AI should produce so you know it worked.

> **Pro tip:** Paste the prompt first, then attach or paste your own code/files in the next message. Most models perform better when the instruction comes before the data.

CONTENTS

Table of Contents

00 Introduction 4

About Web.Lib 5

01 Part 1 — Front-End Development Prompts 01 - 20 6

02 Part 2 — Back-End Development Prompts 21 - 40 27

03 Part 3 — Debugging & Code Review Prompts 41 - 60 48

04 Part 4 — UI/UX & Design Prompts 61 - 80 69

05 Part 5 — Productivity & Career Prompts 81 - 100 90

★ Bonus — Developer Resources 111

25 Best AI Websites 112

25 Best VS Code Extensions 114

25 Best Websites Every Developer Uses 116

25 Best GitHub Repositories 118

✓ Final Words 120

Work With Web.Lib 121

Let's Build Your Next Project 122

✨ INTRODUCTION

Talk to AI like a senior engineer.

AI has quietly become the most powerful tool in a developer's toolbox — but only for the people who know how to talk to it. The difference between a vague "fix my code" and a precise, well-structured prompt is often the difference between ten frustrating minutes and ten useful seconds.

This book exists to close that gap.

Inside you'll find **100 battle-tested prompts** organized around the work developers actually do: building interfaces, designing APIs, hunting bugs, polishing UX, writing documentation, and growing a career. Each one is written the way a senior engineer would brief a sharp junior — with context, clear requirements, modern tooling, and an explicit definition of "done."

These prompts assume a modern stack: **HTML, CSS, JavaScript, TypeScript, React, Next.js, Tailwind CSS, Node.js, Express, Prisma, PostgreSQL, MongoDB, REST, JWT, Docker, Git, and GitHub.** Wherever a prompt references a technology, you can swap it for your own — the structure is what makes it work.

Treat this as a reference, not a novel. Skim the table of contents, find the problem in front of you, copy the prompt, and get back to shipping.

Let's build.

WHO WE ARE

About Web.Lib

Web.Lib is a modern software company that designs and develops high-quality digital products — from polished websites to complex business systems.

We specialize in:

- **Modern Websites**
- **Business Systems**
- **Dashboards**
- **Web Applications**
- **UI/UX Design**

Web.Lib focuses on creating modern, fast and scalable digital experiences that help businesses launch, grow and stand out.

WEBSITE
web-lib-one.vercel.app

PHONE
+218 94 100 0916

SCAN TO VISIT OUR WEBSITE
web-lib-one.vercel.app

PART 01

1

Front-End Development

20 prompts for building fast, accessible, modern interfaces.

Prompts 01 - 20 | 20 prompts

01	Build a Responsive Landing Page	7
02	Create a Reusable React Component Library	8
03	Build a Dark Mode Toggle with Persistence	9
04	Convert a Design Mockup into React + Tailwind	10
05	Build an Accessible Navigation Menu	11
06	Build a Multi-Step Form with Validation	12
07	Build a Developer Portfolio Website	13
08	Optimize React Rendering Performance	14
09	Build a Data Table with Sorting, Filtering & Pagination	15
10	Create a Custom React Hook	16
11	Optimize a Next.js Site for SEO	17
12	Build a Responsive Image Gallery with Lightbox	18
13	Build a Toast Notification System	19
14	Build a Drag-and-Drop Interface	20
15	Build an Admin Panel Layout	21
16	Build Skeleton Loading States	22
17	Build a Form with React Hook Form + Zod	23
18	Build a Responsive Dashboard Layout	24
19	Add Internationalization (i18n) to a React App	25
20	Migrate a JavaScript React Project to TypeScript	26

01

FRONT-END DEVELOPMENT

Build a Responsive Landing Page

WHAT IS THIS PROMPT FOR?

Use this when you need to ship a high-converting marketing or product landing page quickly. It's ideal for freelancers building client sites, indie hackers launching products, or developers who need a polished page without starting from a blank file.

AI PROMPT — copy & paste

PROMPT

```
You are a senior front-end engineer specializing in conversion-focused
marketing sites. Build a complete, responsive landing page using
React, Next.js (App Router) and Tailwind CSS.

Product context:
- Product name: [PRODUCT NAME]
- What it does: [ONE-SENTENCE DESCRIPTION]
- Target audience: [WHO IT'S FOR]
- Primary call to action: [E.G. "Start free trial"]

Requirements:
1. Sections in this order: sticky navbar, hero with headline + subhead +
   primary CTA, social proof/logos, 3-column feature grid, how-it-works
   (3 steps), testimonial, pricing teaser, FAQ accordion, footer.
2. Mobile-first and fully responsive (sm, md, lg breakpoints).
3. Semantic HTML5 and accessible (proper headings, alt text, ARIA on the
   accordion, visible focus states, color contrast ≥ 4.5:1).
4. Use Tailwind only — no external UI libraries. Use a clean, modern
   aesthetic with generous whitespace and a single accent color.
5. Provide reusable components (Navbar, Hero, FeatureCard, etc.) in
   separate code blocks.
6. Include subtle hover/transition effects, no layout shift.

Output the full code, then a short note on where to swap in real content.
```

EXAMPLE RESULT

The AI returns a complete set of React/Next.js components styled with Tailwind, structured into clear sections, mobile-responsive, accessible, and ready to drop into an `app/` directory — plus guidance on replacing placeholder copy.

02

FRONT-END DEVELOPMENT

Create a Reusable React Component Library

WHAT IS THIS PROMPT FOR?

Use this when you keep rewriting the same buttons, inputs, and cards across projects and want a consistent, typed, reusable set of UI primitives. Great for teams standardizing their design language or freelancers reusing code between clients.

AI PROMPT — copy & paste

PROMPT

```
Act as a senior React + TypeScript engineer building a small, reusable UI
component library with Tailwind CSS.

Create the following components with full TypeScript types:
[Button, Input, Textarea, Card, Badge, Modal, Spinner]

For EACH component:
1. Use a typed props interface; extend the correct native HTML element
   props (e.g. ButtonHTMLAttributes) so consumers can pass standard props.
2. Support variants and sizes via a clean prop API (e.g. variant:
   "primary" | "secondary" | "ghost"; size: "sm" | "md" | "lg").
3. Use class-variance-authority or a simple cn() helper for class merging;
   show the helper code too.
4. Forward refs where appropriate (forwardRef).
5. Be fully accessible (correct roles, keyboard support, focus management
   for Modal).

Also provide:
- A usage example for each component.
- A short README snippet documenting the props.

Keep the API minimal and predictable. No external UI kits.
```

EXAMPLE RESULT

A folder's worth of typed, variant-driven components with a shared `cn()` utility, ref forwarding, accessibility built in, usage examples, and lightweight docs — a foundation you can extend across projects.

03

FRONT-END DEVELOPMENT

Build a Dark Mode Toggle with Persistence

WHAT IS THIS PROMPT FOR?

Use this when you need a proper light/dark theme that respects the user's system preference, persists across reloads, and avoids the "flash of wrong theme." Common in dashboards, blogs, and SaaS apps.

AI PROMPT — copy & paste

PROMPT

```
You are a front-end engineer. Implement a complete dark mode system for a
Next.js (App Router) app using Tailwind CSS.

Requirements:
1. Three modes: "light", "dark", and "system" (follows OS preference).
2. Persist the user's choice in localStorage and re-apply it on load.
3. Prevent the flash of incorrect theme (FOUC) by setting the theme class
   before first paint — show the inline script or approach needed.
4. Use Tailwind's "class" dark mode strategy.
5. Provide a reusable <ThemeProvider> (React context) and a <ThemeToggle>
   button component with accessible labels and an icon for each state.
6. Expose a useTheme() hook returning { theme, setTheme }.

Explain where each piece goes in the Next.js project structure, and call
out any caveats with server vs client components.
```

EXAMPLE RESULT

A working theme system: a context provider, a `useTheme` hook, an accessible toggle, Tailwind config tweaks, and an anti-FOUC technique — with notes on placement in a Next.js App Router project.

04

FRONT-END DEVELOPMENT

Convert a Design Mockup into React + Tailwind

WHAT IS THIS PROMPT FOR?

Use this when you have a screenshot, Figma export, or written description of a UI and want clean, responsive code from it. Saves hours of manual translation from design to markup.

AI PROMPT — copy & paste

PROMPT

```
You are an expert at translating designs into pixel-aware, responsive code.
I will describe (or attach) a UI design. Convert it into React + Tailwind CSS.

Design description:
[PASTE DESCRIPTION OR ATTACH IMAGE — include layout, colors, spacing,
fonts, and any interactive elements.]

Requirements:
1. Match the layout, spacing, and visual hierarchy as closely as possible.
2. Use semantic HTML and a mobile-first responsive approach.
3. Break the UI into logical, reusable components.
4. Use Tailwind utility classes only; define any repeated values (colors,
   radii) as comments so I can move them into the Tailwind config.
5. Use placeholder data and clearly mark where real content/props go.
6. Ensure accessibility: alt text, labels, focus states, contrast.

After the code, list any assumptions you made and anything you'd confirm
with the designer.
```

EXAMPLE RESULT

Clean, componentized React + Tailwind markup that faithfully reproduces the described layout, with placeholders, responsive behavior, accessibility, and a list of assumptions to verify.

05

FRONT-END DEVELOPMENT

Build an Accessible Navigation Menu

WHAT IS THIS PROMPT FOR?

Use this when you need a navbar that works for everyone — keyboard users, screen readers, and mobile visitors — including a proper hamburger menu and dropdowns. Accessibility is where most hand-rolled navs fail.

AI PROMPT — copy & paste

PROMPT

```
Act as an accessibility-focused front-end engineer. Build a responsive,
fully accessible navigation bar in React + Tailwind CSS.

Requirements:
1. Desktop: horizontal nav with one dropdown menu containing sub-links.
2. Mobile: collapsible hamburger menu (animated open/close, no layout shift).
3. Full keyboard support: Tab order, Enter/Space to open, Escape to close,
   arrow keys to move within the dropdown.
4. Correct ARIA: aria-expanded, aria-controls, aria-haspopup, role="menu"/
   "menuitem" where appropriate; manage focus when opening/closing.
5. Visible focus indicators and screen-reader-only labels where needed.
6. Trap focus inside the mobile menu when open; return focus to the toggle
   on close.

Provide the component code and a brief checklist I can use to verify
accessibility manually.
```

EXAMPLE RESULT

A responsive navbar with an accessible dropdown and mobile menu, complete keyboard handling, correct ARIA attributes, focus management, and a manual a11y testing checklist.

06

FRONT-END DEVELOPMENT

Build a Multi-Step Form with Validation

WHAT IS THIS PROMPT FOR?

Use this for checkout flows, onboarding wizards, or long sign-up forms where breaking input into steps improves completion rates. Handles state, validation, and navigation between steps.

AI PROMPT — copy & paste

PROMPT

```
You are a senior React engineer. Build a multi-step form ("wizard") in
React + TypeScript + Tailwind, using react-hook-form and zod for validation.

Steps:
1. Account (email, password)
2. Profile (full name, role, company)
3. Review (read-only summary) + submit

Requirements:
1. Validate each step with a zod schema before allowing "Next".
2. Persist data across steps in a single form state; allow "Back" without
   losing input.
3. Show a progress indicator (step X of 3) and per-field error messages.
4. Disable "Next"/"Submit" while invalid or submitting; show a loading state.
5. On final submit, log the assembled payload (placeholder for an API call).
6. Make it accessible: labeled inputs, aria-invalid, focus the first error.

Return the full component(s), the zod schemas, and a short explanation of
the state flow.
```

EXAMPLE RESULT

A working multi-step wizard with per-step zod validation, shared form state, a progress indicator, accessible error handling, and a clear explanation of how data flows between steps.

07

FRONT-END DEVELOPMENT

Build a Developer Portfolio Website

WHAT IS THIS PROMPT FOR?

Use this when you need a personal portfolio that actually lands interviews and clients. Ideal for students, freelancers, and job-seekers who want a professional site without spending a weekend on layout decisions.

AI PROMPT — copy & paste

PROMPT

```
You are a front-end engineer and personal-branding expert. Build a modern,
fast developer portfolio using Next.js + Tailwind CSS.

About me:
- Name / role: [NAME, E.G. "Full-Stack Developer"]
- Top skills: [LIST]
- Notable projects: [2-4 PROJECTS WITH ONE-LINE DESCRIPTIONS]
- Links: [GITHUB, LINKEDIN, EMAIL]

Requirements:
1. Sections: hero (name, tagline, CTA buttons), about, skills grid,
   featured projects (cards with image, stack, links), experience timeline,
   contact section, footer.
2. Responsive, accessible, and SEO-friendly (proper meta tags, semantic
   headings, descriptive alt text).
3. Clean, minimal, professional aesthetic with one accent color and tasteful
   motion (subtle fade/slide on scroll).
4. Project data driven by a typed array so it's easy to update.
5. Include a downloadable-resume button placeholder.

Output the components plus the project data structure I should edit.
```

EXAMPLE RESULT

A complete, responsive portfolio site with structured sections, SEO meta tags, a typed projects array, subtle animations, and clear places to plug in your own details.

08

FRONT-END DEVELOPMENT

Optimize React Rendering Performance

WHAT IS THIS PROMPT FOR?

Use this when a React app feels sluggish — laggy typing, slow lists, janky interactions. The AI audits your component for unnecessary re-renders and expensive work, then proposes targeted fixes.

AI PROMPT — copy & paste

PROMPT

```
Act as a React performance expert. I'll paste a component (or several) that
feels slow. Diagnose and fix the performance problems.

For your analysis:
1. Identify the specific causes of unnecessary re-renders (unstable props,
   inline objects/functions, missing keys, context misuse, etc.).
2. Identify expensive operations running on every render and where to
   memoize them (useMemo, useCallback) — but only where it genuinely helps;
   call out cases where memoization is NOT worth it.
3. Recommend structural fixes: component splitting, lifting/colocating
   state, list virtualization, lazy loading.
4. Provide the refactored code with comments explaining each change.
5. Suggest how to measure the improvement (React DevTools Profiler, what
   metrics to watch).

Here is the code:
[PASTE COMPONENT CODE]
```

EXAMPLE RESULT

A clear diagnosis of re-render and computation problems, a refactored version with memoization and structural improvements applied judiciously, and a plan to measure the speedup with the Profiler.

09

FRONT-END DEVELOPMENT

Build a Data Table with Sorting, Filtering & Pagination

WHAT IS THIS PROMPT FOR?

Use this when you need an interactive table for admin panels or dashboards — the kind of component that's deceptively time-consuming to build well. Handles sorting, search, and paging cleanly.

AI PROMPT — copy & paste

PROMPT

```
You are a senior React + TypeScript engineer. Build a reusable, accessible
data table component with Tailwind CSS.

Features:
1. Generic typing: <DataTable<T>> accepts a typed array and a column config
   (key, header, optional custom cell renderer, sortable flag).
2. Column sorting (asc/desc/none) with visible sort indicators.
3. Global text search/filter across configured columns.
4. Client-side pagination with page size selector and page controls.
5. Empty state, loading state, and a "no results" state.
6. Accessible markup: proper <table> semantics, aria-sort, keyboard-usable
   controls.

Constraints:
- No table libraries — vanilla React + Tailwind.
- Keep it performant for a few thousand rows.

Provide the component, the column config type, and a usage example with
sample data.
```

EXAMPLE RESULT

A generic, typed data table component supporting sort, search, and pagination with proper states and table semantics, plus a usage example you can adapt to any dataset.

10

FRONT-END DEVELOPMENT

Create a Custom React Hook

WHAT IS THIS PROMPT FOR?

Use this when you find repeated stateful logic (fetching, debouncing, local storage, media queries) scattered across components and want to extract it into a clean, reusable, well-typed hook.

AI PROMPT — copy & paste

PROMPT

```
Act as a React + TypeScript expert. Create a production-quality custom hook.

Hook to build: [DESCRIBE — e.g. "useFetch", "useDebounce", "useLocalStorage",
"useMediaQuery", "useClickOutside"]

Requirements:
1. Full TypeScript types with generics where it makes the hook flexible.
2. Handle edge cases (cleanup on unmount, stale closures, SSR safety if
   relevant, race conditions for async hooks).
3. Return a clear, predictable API.
4. Include inline comments explaining non-obvious decisions.
5. Provide a usage example component.
6. Note any gotchas or limitations.

If the hook does async work, ensure it cancels/ignores outdated responses
and exposes loading and error states.
```

EXAMPLE RESULT

A robust, typed custom hook with proper cleanup, edge-case handling, a clean return API, a usage example, and notes on limitations — ready to drop into your project.

11

FRONT-END DEVELOPMENT

Optimize a Next.js Site for SEO

WHAT IS THIS PROMPT FOR?

Use this when your site ranks poorly or doesn't preview well when shared. The AI sets up metadata, structured data, sitemaps, and performance basics so search engines and social platforms understand your pages.

AI PROMPT — copy & paste

PROMPT

```
You are a technical SEO engineer working in Next.js (App Router). Make my
site fully SEO-optimized.

Site context:
- Type: [BLOG / SAAS / PORTFOLIO / ECOMMERCE]
- Main pages: [LIST]
- Primary keywords/topics: [LIST]

Deliver:
1. A metadata strategy using the App Router Metadata API — title templates,
   descriptions, canonical URLs, and per-page overrides.
2. Open Graph and Twitter card tags for rich link previews.
3. JSON-LD structured data appropriate to the site type (e.g. Article,
   Product, BreadcrumbList, Organization).
4. A dynamic sitemap.xml and robots.txt setup.
5. Performance/SEO checklist: image optimization, semantic headings, alt
   text, Core Web Vitals considerations, internal linking.
6. Code examples for the metadata, sitemap, and one JSON-LD block.

Explain what to customize per page and how to verify with Google's tools.
```

EXAMPLE RESULT

A complete SEO setup: Metadata API configuration, OG/Twitter tags, relevant JSON-LD, dynamic sitemap and robots files, and a verification checklist tailored to your site type.

12

FRONT-END DEVELOPMENT

Build a Responsive Image Gallery with Lightbox

WHAT IS THIS PROMPT FOR?

Use this for photography sites, portfolios, or product galleries where users browse images and open them full-size. Handles responsive grids and an accessible lightbox modal.

AI PROMPT — copy & paste

PROMPT

```
You are a front-end engineer. Build a responsive image gallery with a
lightbox in React + Tailwind CSS.

Requirements:
1. Responsive masonry-style or CSS grid layout (1 col mobile → 3-4 cols
   desktop) with lazy-loaded images and proper aspect-ratio handling to
   avoid layout shift.
2. Clicking an image opens an accessible lightbox modal showing the full
   image with next/previous navigation.
3. Lightbox supports keyboard navigation (arrows to move, Escape to close)
   and traps focus while open.
4. Smooth transitions, no scrollbar jump when the modal opens.
5. Driven by a typed array of images (src, alt, width, height).
6. Fully accessible: alt text, aria-label on controls, focus return on close.

Provide the gallery and lightbox components plus the image data type.
```

EXAMPLE RESULT

A responsive, lazy-loaded gallery with an accessible keyboard-navigable lightbox, smooth transitions, no layout shift, and a typed image array you can populate.

13

FRONT-END DEVELOPMENT

Build a Toast Notification System

WHAT IS THIS PROMPT FOR?

Use this when you need app-wide success/error/info messages triggered from anywhere in your code — after saving, on errors, on copy-to-clipboard. A reusable, accessible toast system you call with one function.

AI PROMPT — copy & paste

PROMPT

```
Act as a senior React + TypeScript engineer. Build a reusable toast
notification system with Tailwind CSS — no external libraries.

Requirements:
1. A <ToastProvider> using React context, plus a useToast() hook exposing
   toast.success(), toast.error(), toast.info(), toast.dismiss().
2. Toasts stack, auto-dismiss after a configurable duration, and can be
   dismissed manually; pausing on hover is a plus.
3. Smooth enter/exit animations and a fixed, responsive position (e.g.
   bottom-right on desktop, full-width bottom on mobile).
4. Accessible: use role="status"/aria-live="polite" for info/success and
   an assertive region for errors; dismiss buttons are keyboard-usable.
5. Fully typed API.

Provide the provider, the hook, the toast component, and a usage example
calling it from a button click and from a try/catch.
```

EXAMPLE RESULT

A self-contained toast system: a provider, a typed `useToast` hook, stacked auto-dismissing animated toasts with proper ARIA live regions, and examples of triggering them from anywhere.

14

FRONT-END DEVELOPMENT

Build a Drag-and-Drop Interface

WHAT IS THIS PROMPT FOR?

Use this for kanban boards, sortable lists, or file-reordering UIs. The AI implements smooth, accessible drag-and-drop with the right state updates.

AI PROMPT — copy & paste

PROMPT

```
You are a front-end engineer experienced with interaction design. Build a
drag-and-drop [KANBAN BOARD / SORTABLE LIST] in React + TypeScript + Tailwind.

Use [@dnd-kit/core OR the native HTML Drag and Drop API — recommend which
fits this case and why].

Requirements:
1. Items can be reordered within a list; (for kanban) cards can move between
   columns.
2. Visual feedback: drag handle, drag preview, drop placeholder/indicator.
3. State updates immutably; expose the new order via an onChange callback.
4. Keyboard accessibility for drag-and-drop (move items without a mouse).
5. Smooth transitions and no layout jank.
6. Typed data model for columns/items.

Provide the component(s), the data model, a usage example, and notes on the
accessibility approach.
```

EXAMPLE RESULT

A working drag-and-drop board or list with visual drag feedback, immutable state updates, keyboard support, and a typed data model — plus a recommendation on which DnD approach suits your use case.

15

FRONT-END DEVELOPMENT

Build an Admin Panel Layout

WHAT IS THIS PROMPT FOR?

Use this when starting an internal tool, CMS, or SaaS back office. It scaffolds the classic admin shell — sidebar, topbar, content area — so you can focus on features instead of layout plumbing.

AI PROMPT — copy & paste

PROMPT

```
You are a senior front-end engineer. Build a responsive admin panel layout
in Next.js (App Router) + Tailwind CSS.

Requirements:
1. Layout shell: collapsible left sidebar (nav with icons + labels, active
   state, grouped sections), sticky top bar (search, notifications, user
   menu dropdown), and a main content area with a page header + breadcrumbs.
2. Responsive: sidebar collapses to icons on tablet and becomes an overlay
   drawer on mobile.
3. Reusable building blocks: <StatCard>, <DataSection>, page <Header>.
4. A sample dashboard page using these blocks (4 stat cards + a placeholder
   chart area + a recent-activity list).
5. Accessible navigation (keyboard, aria-current on active link, focus
   management for the mobile drawer).
6. Clean, professional, neutral styling.

Provide the layout components, the sample page, and the folder structure.
```

EXAMPLE RESULT

A reusable admin shell — responsive collapsible sidebar, top bar with user menu, breadcrumbs, and reusable stat/section components — plus a sample dashboard page and a suggested folder structure.

16

FRONT-END DEVELOPMENT

Build Skeleton Loading States

WHAT IS THIS PROMPT FOR?

Use this to replace jarring spinners and blank screens with polished skeleton placeholders that match your content's shape, improving perceived performance while data loads.

AI PROMPT — copy & paste

PROMPT

```
Act as a front-end engineer focused on perceived performance. Create a
reusable skeleton-loading system in React + Tailwind CSS.

Requirements:
1. A base <Skeleton> primitive (configurable width, height, radius) with a
   subtle shimmer/pulse animation using Tailwind.
2. Composed skeletons that mirror real components: SkeletonCard,
   SkeletonTableRow, SkeletonAvatar+text line, SkeletonList.
3. Respect prefers-reduced-motion (disable shimmer for those users).
4. Show how to swap between skeleton and real content based on a loading
   boolean, avoiding layout shift between the two states.
5. Accessible: mark skeletons as aria-hidden and expose an aria-busy/status
   region for screen readers.

Provide the primitives, the composed skeletons, and a usage example for a
card grid that's loading.
```

EXAMPLE RESULT

A skeleton primitive plus content-shaped composites with a reduced-motion-friendly shimmer, correct ARIA handling, and an example showing a smooth swap from skeleton to real content.

17

FRONT-END DEVELOPMENT

Build a Form with React Hook Form + Zod

WHAT IS THIS PROMPT FOR?

Use this for any non-trivial form where you want robust validation, great DX, and minimal re-renders. This is the modern standard combo for forms in React.

AI PROMPT — copy & paste

PROMPT

```
You are a React + TypeScript expert. Build a fully validated form using
react-hook-form and zod with Tailwind styling.

Form: [DESCRIBE FIELDS — e.g. "sign up: name, email, password, confirm
password, terms checkbox"]

Requirements:
1. Define a single zod schema as the source of truth; infer the TS type from
   it (z.infer).
2. Use zodResolver with react-hook-form; show per-field inline errors and
   handle cross-field validation (e.g. password === confirmPassword).
3. Disable submit while invalid or submitting; show a submitting state.
4. Accessible: associate labels, set aria-invalid and aria-describedby for
   errors, focus the first invalid field on submit.
5. On submit, validate then log/return the typed data (placeholder for API).
6. Keep components clean and reusable (a small <FormField> wrapper is fine).

Provide the schema, the form component, and the wrapper.
```

EXAMPLE RESULT

A complete form wired to a single zod schema with inferred types, inline and cross-field validation, accessible error handling, and a clean reusable field wrapper.

18

FRONT-END DEVELOPMENT

Build a Responsive Dashboard Layout

WHAT IS THIS PROMPT FOR?

Use this when you need an analytics or metrics dashboard with cards, charts, and responsive grids. Focuses on the layout and composition so you can plug in real data and chart libraries later.

AI PROMPT — copy & paste

PROMPT

```
You are a front-end engineer. Build a responsive analytics dashboard UI in
React + Tailwind CSS.

Requirements:
1. A responsive grid: top row of 4 KPI stat cards (label, value, delta vs
   last period with up/down color), then a 2/3 + 1/3 split for a main chart
   area and a side panel (e.g. top items list), then a full-width data table
   placeholder.
2. Each section is a reusable component with a header and optional actions.
3. Responsive behavior: cards stack on mobile, grid reflows at breakpoints.
4. Include a charts placeholder component with a comment on where to wire in
   Recharts/Chart.js.
5. Accessible and keyboard-navigable; meaningful headings and labels.
6. Clean, data-dense but readable styling.

Provide the layout, the reusable components, and sample placeholder data.
```

EXAMPLE RESULT

A responsive dashboard composed of reusable KPI cards, a chart area, a side list, and a table placeholder, with sample data and clear hooks for adding a real charting library.

19

FRONT-END DEVELOPMENT

Add Internationalization (i18n) to a React App

WHAT IS THIS PROMPT FOR?

Use this when you need to support multiple languages — translating UI text, handling locale formatting, and letting users switch languages. Essential for products expanding to new markets.

AI PROMPT — copy & paste

PROMPT

```
Act as an internationalization expert. Add i18n to my [REACT / NEXT.JS]
application.

Languages to support: [E.G. English, Spanish, Arabic]

Deliver:
1. A recommended library for my framework (e.g. next-intl or react-i18next)
   and why.
2. Project structure for translation files (JSON per locale) and a clean
   key naming convention.
3. Setup code: provider/config, a useTranslation-style hook, and a language
   switcher component that persists the choice.
4. Examples of: simple strings, interpolation (variables), pluralization,
   and date/number formatting via Intl.
5. RTL support guidance if any language requires it (e.g. Arabic) — how to
   flip layout direction.
6. How to handle missing translations gracefully.

Show the config, two sample locale files, and a translated component.
```

EXAMPLE RESULT

A complete i18n setup with a recommended library, organized locale files, a persistent language switcher, examples of interpolation/pluralization/formatting, and RTL guidance.

20

FRONT-END DEVELOPMENT

Migrate a JavaScript React Project to TypeScript

WHAT IS THIS PROMPT FOR?

Use this when you have a working JS React codebase and want type safety without a risky big-bang rewrite. The AI gives you a safe, incremental migration plan and converts files correctly.

AI PROMPT — copy & paste

PROMPT

```
You are a TypeScript migration specialist. Help me migrate a React project
from JavaScript to TypeScript incrementally and safely.

Provide:
1. A recommended tsconfig.json for a gradual migration (allowJs, strict
   settings to enable over time) with comments.
2. A step-by-step migration order (config → rename leaf components → shared
   utils → context/state → entry points) and why this order reduces risk.
3. How to type: component props, useState, useRef, event handlers, context,
   and API response data.
4. How to handle third-party libs without types (@types packages or
   declaration files).
5. Common migration errors and how to fix them.

Then convert this file as a worked example:
[PASTE A .JS/.JSX FILE]

Return the typed version with comments on each meaningful type added.
```

EXAMPLE RESULT

A migration-ready `tsconfig`, a low-risk step-by-step plan, typing guidance for common React patterns, and a fully converted example file with explained types.

PART 02

2

Back-End Development

20 prompts for designing robust APIs, databases, and authentication.

Prompts 21 - 40

20 prompts

21	Design a RESTful API	28
22	Build JWT Authentication with Node & Express	29
23	Design a PostgreSQL Schema with Prisma	30
24	Build a CRUD API with Express & Prisma	31
25	Implement Role-Based Access Control (RBAC)	32
26	Build Rate Limiting Middleware	33
27	Design a MongoDB Schema with Mongoose	34
28	Build a Secure File Upload Endpoint	35
29	Add Pagination, Filtering & Sorting to an API	36
30	Implement a Refresh Token Flow	37
31	Build Reusable Validation Middleware	38
32	Build a Background Job Queue	39
33	Design a Microservices Architecture	40
34	Implement Caching with Redis	41
35	Build a Webhook Handler	42
36	Create a Database Migration Strategy	43
37	Build an Email Sending Service	44
38	Implement API Versioning	45
39	Build a GraphQL API	46
40	Harden an Express App for Production	47

21

BACK-END DEVELOPMENT

Design a RESTful API

WHAT IS THIS PROMPT FOR?

Use this when starting a new service and you want clean, predictable, well-versioned REST endpoints before writing any code. Avoids the inconsistent, hard-to-maintain APIs that come from designing as you go.

AI PROMPT — copy & paste

PROMPT

```
You are a senior backend architect. Design a clean, RESTful API for the
following domain.

Domain: [DESCRIBE — e.g. "a task management app with users, projects, tasks"]
Stack: Node.js + Express + Prisma + PostgreSQL

Deliver:
1. The resource model and relationships (list entities and how they relate).
2. A full endpoint table: method, path, purpose, auth required?, request
   body shape, success response, status codes. Follow REST conventions
   (plural nouns, nesting only where it makes sense, proper verbs).
3. Consistent JSON response envelope and a standard error format.
4. Pagination, filtering, and sorting conventions for list endpoints.
5. Versioning strategy (e.g. /api/v1) and naming conventions.
6. Auth approach at a high level (which endpoints are protected).

Present the endpoints as a clear table, then note 3 design trade-offs you made.
```

EXAMPLE RESULT

A complete API blueprint: an entity model, a tidy endpoint table with verbs/paths/status codes, a consistent response and error format, pagination conventions, and documented trade-offs.

22

BACK-END DEVELOPMENT

Build JWT Authentication with Node & Express

WHAT IS THIS PROMPT FOR?

Use this when you need secure login/registration with JSON Web Tokens. Covers hashing, token issuing, and protecting routes — the foundation of most app authentication.

AI PROMPT — copy & paste

PROMPT

```
You are a backend security engineer. Implement JWT authentication in
Node.js + Express + TypeScript with Prisma + PostgreSQL.

Requirements:
1. Endpoints: POST /auth/register, POST /auth/login, GET /auth/me (protected).
2. Hash passwords with bcrypt (correct salt rounds); never store plaintext.
3. Issue a signed JWT on login containing minimal claims (user id, role);
   set a sensible expiry. Read the secret from an env var.
4. An authenticate middleware that verifies the token, attaches the user to
   req, and returns 401 on failure.
5. Input validation (zod) and a consistent error format; never leak whether
   an email exists on failed login.
6. Notes on storing the token securely on the client (httpOnly cookie vs
   header) and the trade-offs.

Provide the Prisma user model, the routes, the middleware, and the validation
schemas.
```

EXAMPLE RESULT

A working auth flow: a Prisma user model, register/login/me endpoints, bcrypt hashing, JWT issuing and verification middleware, validation, and security notes on token storage.

23

BACK-END DEVELOPMENT

Design a PostgreSQL Schema with Prisma

WHAT IS THIS PROMPT FOR?

Use this when you need a normalized, performant database schema for a new feature or app. The AI turns your requirements into a proper Prisma schema with correct relations and indexes.

AI PROMPT — copy & paste

PROMPT

```
Act as a database architect. Design a PostgreSQL schema using Prisma for
the following application.

Requirements: [DESCRIBE THE APP AND KEY ENTITIES — e.g. "blog with users,
posts, comments, tags, likes"]

Deliver:
1. A complete schema.prisma with models, fields, correct types, and
   relations (one-to-one, one-to-many, many-to-many with explicit join
   tables where needed).
2. Sensible constraints: required vs optional, @unique, default values,
   created/updated timestamps, soft-delete field if appropriate.
3. Indexes on columns used for lookups/filtering/foreign keys, with a note
   on why each index exists.
4. Enums where they fit; cascade rules on relations.
5. A short explanation of the normalization choices and any intentional
   denormalization for performance.
6. 2-3 example queries (Prisma client) for common access patterns.

Flag any decisions that depend on expected scale or read/write patterns.
```

EXAMPLE RESULT

A complete `schema.prisma` with well-typed models, proper relations, constraints and indexes (each justified), enums, cascade rules, and example queries — plus notes on normalization trade-offs.

24

BACK-END DEVELOPMENT

Build a CRUD API with Express & Prisma

WHAT IS THIS PROMPT FOR?

Use this to scaffold complete create/read/update/delete endpoints for a resource quickly, with validation and error handling baked in — so you're not hand-writing boilerplate every time.

AI PROMPT — copy & paste

PROMPT

```
You are a senior Node.js engineer. Build a complete CRUD API for a single
resource using Express + TypeScript + Prisma + PostgreSQL.

Resource: [E.G. "Product" with fields name, description, price, stock,
categoryId]

Requirements:
1. Endpoints: list (with pagination, filtering, sorting), get one, create,
   update, delete. Use correct HTTP methods and status codes.
2. Layered structure: routes → controller → service → Prisma. Keep business
   logic out of the controllers.
3. Validate request bodies and query params with zod.
4. Centralized error handling middleware returning a consistent error shape;
   handle "not found" (404) and validation (400) cleanly.
5. Async errors caught properly (wrapper or try/catch).
6. Type-safe throughout.

Provide the folder structure, the route/controller/service files, the
validation schemas, and the error middleware.
```

EXAMPLE RESULT

A cleanly layered CRUD API (routes/controller/service/Prisma) with pagination and filtering, zod validation, centralized error handling, correct status codes, and a clear folder structure.

25

BACK-END DEVELOPMENT

Implement Role-Based Access Control (RBAC)

WHAT IS THIS PROMPT FOR?

Use this when different users need different permissions — admins, editors, viewers. The AI builds a flexible authorization layer so you can protect routes and actions by role.

AI PROMPT — copy & paste

PROMPT

```
Act as a backend security engineer. Implement role-based access control
(RBAC) for an Express + TypeScript API that already uses JWT auth.

Requirements:
1. Define roles and a permissions model: [E.G. roles admin, editor, viewer;
   permissions like post:create, post:delete, user:manage]. Recommend
   role→permission mapping vs direct permission checks and justify it.
2. An authorize() middleware that checks the authenticated user's role/
   permissions against what a route requires; returns 403 on failure.
3. Support resource-level checks (e.g. a user can edit only their own posts)
   in addition to role checks.
4. Keep it declarative — protecting a route should be one readable line.
5. Avoid common pitfalls (privilege escalation, missing checks on nested
   routes, trusting client-sent roles).

Provide the roles/permissions config, the middleware, example protected
routes, and an ownership-check example.
```

EXAMPLE RESULT

A flexible RBAC layer: a roles/permissions config, a declarative `authorize()` middleware, resource-ownership checks, and protected route examples — with the common security pitfalls called out.

26

BACK-END DEVELOPMENT

Build Rate Limiting Middleware

WHAT IS THIS PROMPT FOR?

Use this to protect your API from abuse, brute-force attacks, and accidental overload. Especially important for auth endpoints and public APIs.

AI PROMPT — copy & paste

PROMPT

```
You are a backend engineer hardening an Express + TypeScript API. Implement
rate limiting.

Requirements:
1. Recommend an approach: in-memory for a single instance vs Redis-backed
   for multiple instances — explain when to use each.
2. Implement a configurable limiter (max requests per window per key) where
   the key is IP by default but can be user id for authenticated routes.
3. Apply stricter limits to sensitive endpoints (login, password reset) and
   looser limits elsewhere; show how to compose multiple limiters.
4. Return proper 429 responses with Retry-After and rate-limit headers.
5. Make limits configurable via env vars.

Provide the middleware (both the simple and the Redis-backed version),
example usage on a login route, and notes on bypassing limits for trusted
clients.
```

EXAMPLE RESULT

Configurable rate-limiting middleware in both in-memory and Redis-backed forms, with per-route limits, correct `429` / `Retry-After` headers, and guidance on when to use each approach.

27

BACK-END DEVELOPMENT

Design a MongoDB Schema with Mongoose

WHAT IS THIS PROMPT FOR?

Use this when building on MongoDB and you want a thoughtful document model — knowing when to embed vs reference, and how to index for your access patterns.

AI PROMPT — copy & paste

PROMPT

```
Act as a MongoDB data modeling expert. Design a schema for the following
app using Mongoose (Node.js + TypeScript).

App: [DESCRIBE ENTITIES AND MAIN ACCESS PATTERNS — what gets read/written
most]

Deliver:
1. The collections and Mongoose schemas with proper types and validation.
2. For each relationship, decide embed vs reference and JUSTIFY it based on
   access patterns, document growth, and update frequency.
3. Indexes (including compound indexes) tuned to the main queries, each with
   a reason.
4. Sensible schema options: timestamps, default values, required fields,
   enums.
5. TypeScript interfaces for the documents.
6. 2-3 example queries showing the intended access patterns.

Explicitly call out where the model is optimized for reads vs writes and any
trade-offs.
```

EXAMPLE RESULT

Mongoose schemas with typed interfaces, justified embed-vs-reference decisions, access-pattern-driven indexes, sensible options, and example queries — with read/write trade-offs made explicit.

28

BACK-END DEVELOPMENT

Build a Secure File Upload Endpoint

WHAT IS THIS PROMPT FOR?

Use this when users need to upload images, documents, or avatars. File uploads are a common security hole — this prompt covers validation, size limits, and safe storage.

AI PROMPT — copy & paste

PROMPT

```
You are a backend security engineer. Implement a secure file upload endpoint
in Express + TypeScript.

Requirements:
1. Use multer (or recommend a better fit) to accept [SINGLE / MULTIPLE]
   files of type [E.G. images: jpg, png, webp].
2. Validate strictly: allowed MIME types AND file extension AND magic-byte/
   content check (don't trust the client-sent MIME), max file size, max
   number of files.
3. Generate safe, unique filenames (no path traversal, no user-controlled
   names); never execute or serve uploads from a code path.
4. Show two storage strategies: local disk and uploading to cloud storage
   (e.g. S3-compatible) — and which to prefer for production.
5. Return a clean response with the stored file URL/metadata; handle errors
   (too large, wrong type) with proper status codes.
6. Note virus-scanning and access-control considerations.

Provide the route, the validation/config, and the error handling.
```

EXAMPLE RESULT

A hardened upload endpoint with strict type/size validation, safe filename generation, both local and cloud storage strategies, proper error responses, and notes on scanning and access control.

29

BACK-END DEVELOPMENT

Add Pagination, Filtering & Sorting to an API

WHAT IS THIS PROMPT FOR?

Use this when a list endpoint returns too much data and clients need to page, filter, and sort it. Establishes consistent, reusable query conventions across your API.

AI PROMPT — copy & paste

PROMPT

```
Act as a senior backend engineer. Add robust pagination, filtering, and
sorting to a list endpoint in Express + Prisma + TypeScript.

Resource: [E.G. "GET /products"]

Requirements:
1. Support both offset (page/limit) AND cursor-based pagination — explain
   when each is appropriate and implement the one best suited here, noting
   how to switch.
2. Filtering via query params with an allowlist of filterable fields
   (prevent arbitrary/unsafe filters); support common operators (equals,
   contains, gt/lt, in).
3. Sorting via query param with an allowlist of sortable fields and asc/desc.
4. Validate and sanitize all query params with zod; apply sane defaults and
   max limits.
5. Return a consistent response: data array + meta (total, page, pageSize,
   hasNext / nextCursor).
6. Translate the validated params into a Prisma query safely.

Provide the validation schema, the query builder, and the controller.
```

EXAMPLE RESULT

A reusable list endpoint supporting safe filtering/sorting allowlists and pagination (offset and cursor), validated query params with defaults and caps, and a consistent `data + meta` response.

30

BACK-END DEVELOPMENT

Implement a Refresh Token Flow

WHAT IS THIS PROMPT FOR?

Use this when short-lived access tokens need to be renewed without forcing users to log in repeatedly. Covers secure refresh token storage, rotation, and revocation.

AI PROMPT — copy & paste

PROMPT

```
You are a backend security engineer. Implement a secure access + refresh
token flow for an Express + TypeScript + Prisma API.

Requirements:
1. Short-lived access token (JWT) + longer-lived refresh token. Store the
   refresh token in an httpOnly, Secure, SameSite cookie; explain why.
2. Endpoints: POST /auth/login (issues both), POST /auth/refresh (rotates
   the refresh token and issues a new access token), POST /auth/logout
   (revokes the refresh token).
3. Implement refresh token ROTATION and detection of reuse (revoke the whole
   session/family if a used token is replayed).
4. Persist refresh tokens (or their hashes) in the DB so they can be revoked;
   never store them in plaintext.
5. Handle expiry, invalid, and revoked tokens with proper status codes.
6. Note CSRF considerations given cookie usage.

Provide the Prisma model for tokens, the three endpoints, and the rotation/
reuse-detection logic.
```

EXAMPLE RESULT

A secure token flow with short-lived access tokens, httpOnly refresh cookies, rotation with reuse detection, DB-backed revocation, and CSRF notes — plus the endpoints and token model.

31

BACK-END DEVELOPMENT

Build Reusable Validation Middleware

WHAT IS THIS PROMPT FOR?

Use this to validate request bodies, params, and query strings consistently across all routes with a single, reusable mechanism — eliminating scattered, inconsistent validation.

AI PROMPT — copy & paste

PROMPT

```
Act as a Node.js + TypeScript engineer. Create reusable request-validation
middleware for Express using zod.

Requirements:
1. A validate() higher-order middleware that accepts a schema for body,
   params, and/or query and validates the matching parts of the request.
2. On failure, return 400 with a structured error listing each invalid
   field and message; on success, attach the parsed/typed data to the
   request for downstream handlers.
3. Strip unknown fields and coerce types where appropriate (e.g. numeric
   query params) safely.
4. Make the typing strong so controllers get fully typed req.body/params/
   query.
5. Provide a couple of example schemas and a route using the middleware.

Explain how this centralizes validation and keeps controllers clean.
```

EXAMPLE RESULT

A reusable `validate()` middleware that checks body/params/query against zod schemas, returns structured 400 errors, strips unknown fields, and gives controllers fully typed inputs — with examples.

32

BACK-END DEVELOPMENT

Build a Background Job Queue

WHAT IS THIS PROMPT FOR?

Use this when slow tasks (sending emails, processing images, generating reports) shouldn't block the request. Offloads work to a queue processed by background workers.

AI PROMPT — copy & paste

PROMPT

```
You are a backend engineer. Design and implement a background job system for
a Node.js + TypeScript app.

Use case: [E.G. "send welcome emails and generate PDF reports without
blocking the request"]

Deliver:
1. A recommended queue (e.g. BullMQ with Redis) and why it fits.
2. Producer code: how an API route enqueues a job with a typed payload.
3. Worker code: a separate process that consumes jobs, with concurrency
   control.
4. Reliability: retries with exponential backoff, max attempts, and a
   dead-letter/failed-jobs strategy; make jobs idempotent.
5. Observability: logging job lifecycle and how to monitor the queue.
6. Graceful shutdown so in-flight jobs aren't lost on deploy.

Provide the queue setup, a producer example, a worker example, and notes on
running the worker as a separate process.
```

EXAMPLE RESULT

A complete job-queue setup with typed producers, a separate worker process, retries with backoff, idempotency, failed-job handling, monitoring notes, and graceful shutdown.

33

BACK-END DEVELOPMENT

Design a Microservices Architecture

WHAT IS THIS PROMPT FOR?

Use this when an app has outgrown a single codebase and you're considering splitting it into services. The AI helps you decide service boundaries and how they communicate — including whether you should split at all.

AI PROMPT — copy & paste

PROMPT

```
Act as a software architect. Help me design a microservices architecture for
the following system — and first tell me honestly whether microservices are
even the right call versus a modular monolith.

System: [DESCRIBE THE APP, TEAM SIZE, SCALE, AND PAIN POINTS]

Deliver:
1. A recommendation: microservices vs modular monolith, with reasoning based
   on my context. Don't default to microservices.
2. If microservices: proposed service boundaries (by business capability),
   what data each owns, and why these boundaries.
3. Communication patterns: sync (REST/gRPC) vs async (message/event queue),
   and where to use each.
4. How to handle cross-service data, transactions (saga pattern), and
   eventual consistency.
5. Cross-cutting concerns: auth, API gateway, service discovery, logging/
   tracing.
6. Key risks and a phased migration path from the current setup.

Be pragmatic and call out where microservices would add unjustified
complexity.
```

EXAMPLE RESULT

An honest architecture recommendation (often "not yet"), and if warranted, service boundaries, communication patterns, data/consistency strategy, cross-cutting concerns, and a phased migration plan.

34

BACK-END DEVELOPMENT

Implement Caching with Redis

WHAT IS THIS PROMPT FOR?

Use this when expensive database queries or external API calls slow down your API. Adds a caching layer to cut latency and load — with the invalidation strategy that usually gets overlooked.

AI PROMPT — copy & paste

PROMPT

```
You are a backend performance engineer. Add Redis caching to an Express +
TypeScript + Prisma API.

Target: [E.G. "an expensive GET /products endpoint that hits multiple tables"]

Deliver:
1. A cache-aside (lazy loading) implementation: check cache → on miss, query
   DB → store in cache with a TTL → return.
2. A sensible, collision-free key naming scheme that includes query params.
3. An invalidation strategy: which writes invalidate which keys, and how to
   do it reliably (avoid stale data). Discuss TTL vs explicit invalidation.
4. Protection against cache stampede (e.g. locking or jittered TTLs).
5. A reusable cache helper/wrapper so caching any function is one call.
6. Notes on what NOT to cache and the consistency trade-offs.

Provide the Redis client setup, the reusable wrapper, a cached endpoint, and
the invalidation on the related write.
```

EXAMPLE RESULT

A Redis cache-aside layer with a clean key scheme, a reusable caching wrapper, a concrete invalidation strategy on writes, stampede protection, and guidance on consistency trade-offs.

35

BACK-END DEVELOPMENT

Build a Webhook Handler

WHAT IS THIS PROMPT FOR?

Use this when integrating with services like Stripe, GitHub, or payment providers that notify your app via webhooks. Covers signature verification, idempotency, and reliable processing.

AI PROMPT — copy & paste

PROMPT

```
Act as a backend engineer. Implement a secure, reliable webhook receiver in
Express + TypeScript for [E.G. Stripe / GitHub / a generic provider].

Requirements:
1. Verify the webhook signature using the provider's secret BEFORE trusting
   the payload (use the raw request body — show how to configure the body
   parser correctly for this).
2. Respond quickly with 2xx to acknowledge receipt, then process
   asynchronously (enqueue) so slow processing doesn't cause retries.
3. Idempotency: dedupe events by their event id so retries don't double-
   process.
4. Handle different event types via a clean switch/handler map.
5. Robust error handling and logging; return appropriate status codes so the
   provider retries only when it should.
6. Notes on testing locally (e.g. CLI tunneling / provider test events).

Provide the route with signature verification, the handler map, and the
idempotency check.
```

EXAMPLE RESULT

A secure webhook endpoint with raw-body signature verification, fast acknowledgement plus async processing, idempotent event handling, a handler map, and local testing notes.

36

BACK-END DEVELOPMENT

Create a Database Migration Strategy

WHAT IS THIS PROMPT FOR?

Use this when schema changes keep breaking environments or causing downtime. The AI defines a safe, repeatable migration workflow — including how to make breaking changes without downtime.

AI PROMPT — copy & paste

PROMPT

```
You are a database/DevOps engineer. Define a safe database migration strategy
for a [POSTGRESQL + PRISMA] app deployed to production.

Deliver:
1. A migration workflow: how migrations are created, reviewed, versioned in
   Git, and applied in dev/staging/prod (including in CI/CD).
2. The expand-and-contract (parallel change) pattern for breaking changes
   (e.g. renaming a column, splitting a table) so deploys cause zero
   downtime — with a concrete worked example.
3. How to handle data backfills for large tables safely (batching).
4. Rollback strategy and how to recover from a failed migration.
5. Guardrails: never edit applied migrations, how to handle migration drift,
   and reviewing migrations in PRs.
6. A checklist to run before applying a migration to production.

Use a concrete example like "rename users.name to users.full_name with no
downtime".
```

EXAMPLE RESULT

A safe migration workflow covering versioning, CI/CD application, the expand-and-contract pattern with a worked zero-downtime example, batched backfills, rollback steps, and a pre-deploy checklist.

37

BACK-END DEVELOPMENT

Build an Email Sending Service

WHAT IS THIS PROMPT FOR?

Use this when your app needs to send transactional emails — verification, password resets, receipts — reliably and with maintainable templates instead of inline HTML strings.

AI PROMPT — copy & paste

PROMPT

```
You are a backend engineer. Build a reusable transactional email service in
Node.js + TypeScript.

Requirements:
1. An abstraction (interface) so the provider (e.g. Nodemailer/SMTP,
   Resend, SES) can be swapped without changing calling code.
2. A typed sendEmail() and typed helpers per email type (e.g.
   sendVerificationEmail(user, token)).
3. Templating: keep HTML templates separate from logic; support variables
   and a plain-text fallback.
4. Reliability: send asynchronously via a queue/worker so requests aren't
   blocked; retries on transient failures.
5. Config from env vars; a dev mode that logs emails instead of sending.
6. Error handling and logging.

Provide the interface, one provider implementation, a template example, and
a typed helper, plus how to call it from a route.
```

EXAMPLE RESULT

A provider-agnostic email service with typed helpers, separated templates with text fallbacks, async sending via a queue, a dev log mode, and clear usage from your routes.

38

BACK-END DEVELOPMENT

Implement API Versioning

WHAT IS THIS PROMPT FOR?

Use this when you need to evolve a public or shared API without breaking existing clients. The AI sets up a versioning strategy and a deprecation path.

AI PROMPT — copy & paste

PROMPT

```
Act as an API architect. Design and implement an API versioning strategy for
an Express + TypeScript API.

Deliver:
1. Compare versioning approaches (URI path /v1, header-based, query param)
   and recommend one for a public REST API, with reasoning.
2. Implement the chosen approach: folder/router structure that keeps v1 and
   v2 isolated while sharing common logic where safe.
3. A clear policy for what counts as a breaking vs non-breaking change.
4. A deprecation workflow: how to signal deprecation (headers, docs,
   timeline) and sunset an old version gracefully.
5. How to avoid duplicating logic across versions (shared services,
   version-specific controllers/serializers).
6. An example: v1 and v2 of the same endpoint where the response shape
   changed.

Provide the router structure and the example endpoint in both versions.
```

EXAMPLE RESULT

A recommended versioning approach with an isolated-yet-DRY router structure, a breaking-change policy, a deprecation/sunset workflow, and a worked v1-vs-v2 endpoint example.

39

BACK-END DEVELOPMENT

Build a GraphQL API

WHAT IS THIS PROMPT FOR?

Use this when clients need flexible queries and you want to avoid over- and under-fetching common with REST. Covers schema design, resolvers, and the N+1 problem.

AI PROMPT — copy & paste

PROMPT

```
You are a backend engineer experienced with GraphQL. Build a GraphQL API in
Node.js + TypeScript (Apollo Server or GraphQL Yoga) backed by Prisma.

Domain: [DESCRIBE ENTITIES AND RELATIONSHIPS]

Deliver:
1. A typed GraphQL schema (SDL): types, queries, mutations, inputs, and
   relationships.
2. Resolvers wired to Prisma, with proper typing.
3. Solve the N+1 query problem (DataLoader or Prisma's relation loading) —
   explain the issue and the fix.
4. Auth/context: how to pass the authenticated user into resolvers and
   protect specific fields/mutations.
5. Error handling and input validation.
6. Pagination on list fields (cursor-based / Relay-style or simple).

Provide the schema, key resolvers, the DataLoader setup, and the context/auth
wiring. Note when GraphQL is a worse fit than REST.
```

EXAMPLE RESULT

A typed GraphQL schema with Prisma-backed resolvers, an N+1 fix via DataLoader, auth via context, validation, pagination, and an honest note on when REST would serve better.

40

BACK-END DEVELOPMENT

Harden an Express App for Production

WHAT IS THIS PROMPT FOR?

Use this before shipping a Node.js API to production. The AI runs through the essential security and reliability hardening that's easy to forget until something breaks.

AI PROMPT — copy & paste

PROMPT

```
Act as a backend security engineer. Review and harden my Express + TypeScript
API for production. Give me a prioritized hardening checklist with code.

Cover:
1. Security headers (helmet), strict CORS configuration (allowlist origins),
   and disabling fingerprinting (x-powered-by).
2. Input validation everywhere, payload size limits, and protection against
   common injection (parameterized queries via Prisma, no string-built SQL).
3. Rate limiting and brute-force protection on auth routes.
4. Secrets management (env vars, never committed) and secure cookie flags.
5. Centralized error handling that never leaks stack traces/internal details
   in responses; structured logging instead.
6. Graceful shutdown, request timeouts, and health-check endpoints.
7. Dependency hygiene (audit, lockfile) and HTTPS/TLS assumptions behind a
   proxy.

Present it as an ordered checklist (most critical first) with a short code
snippet for each item, then anything I should add to CI.
```

EXAMPLE RESULT

A prioritized production-hardening checklist — headers, CORS, validation, rate limiting, secrets, safe error handling, graceful shutdown, health checks, dependency hygiene — each with a concrete snippet.

PART 03

3

Debugging & Code Review

20 prompts to find bugs faster, understand code, and review like a senior.

Prompts 41 - 60

20 prompts

41 Debug a Runtime Error from a Stack Trace 49

42 Explain Unfamiliar Code 50

43 Perform a Senior-Level Code Review 51

44 Find and Fix a Memory Leak 52

45 Debug an Async / Promise Issue 53

46 Fix a React Re-render or Infinite Loop Bug 54

47 Diagnose and Fix a CORS Error 55

48 Debug a Failing Test 56

49 Find and Fix a Race Condition 57

50 Review Code for Security Vulnerabilities 58

51 Optimize a Slow Database Query 59

52 Decode a Confusing Error Message 60

53 Refactor Legacy Code Safely 61

54 Find the Root Cause (Root Cause Analysis) 62

55 Debug a Production Incident 63

56 Review a Pull Request 64

57 Identify and Fix Code Smells 65

58 Fix a Build or Compilation Error 66

59 Reproduce an Intermittent Bug 67

60 Compare Two Approaches (Trade-off Analysis) 68

41

DEBUGGING & CODE REVIEW

Debug a Runtime Error from a Stack Trace

WHAT IS THIS PROMPT FOR?

Use this when your app crashes and you have an error message and stack trace but aren't sure what's actually wrong. The AI traces it back to the root cause instead of guessing.

AI PROMPT — copy & paste

PROMPT

```
You are a senior engineer and expert debugger. I have a runtime error.
Diagnose the root cause and give me a fix.

Error message + stack trace:
[PASTE FULL ERROR AND STACK TRACE]

Relevant code:
[PASTE THE CODE NEAR THE FAILING LINE]

Context: [LANGUAGE/FRAMEWORK, WHEN IT HAPPENS, WHAT I EXPECTED]

Walk through this systematically:
1. Explain in plain language what the error actually means.
2. Read the stack trace and identify the exact line and call path where it
   originates (not just where it surfaces).
3. State the most likely root cause and 1-2 alternative causes, ranked.
4. Give the corrected code with the change explained.
5. Suggest how to prevent this class of bug in future (a guard, a type, a
   test).

If you need more info to be sure, tell me exactly what to check.
```

EXAMPLE RESULT

A plain-English explanation of the error, the precise origin in the call path, a ranked root-cause diagnosis, corrected code, and a prevention tip — or a specific request for more information.

42

DEBUGGING & CODE REVIEW

Explain Unfamiliar Code

WHAT IS THIS PROMPT FOR?

Use this when you inherit a codebase, read a confusing function, or study someone else's solution and need to understand exactly what it does and why.

AI PROMPT — copy & paste

PROMPT

```
Act as a patient senior engineer mentoring me. Explain the following code so
I fully understand it.

Code:
[PASTE CODE]

Please:
1. Give a one-paragraph high-level summary of what this code does and why it
   exists.
2. Walk through it step by step, explaining each significant block and any
   non-obvious lines.
3. Clarify any tricky language features, patterns, or idioms used.
4. Trace what happens with a concrete example input → output.
5. Point out any bugs, edge cases, or code smells you notice.
6. Suggest how it could be made clearer or more robust (without changing
   behavior).

Assume I know the basics of the language but not this codebase. Avoid jargon
without defining it.
```

EXAMPLE RESULT

A high-level summary, a step-by-step walkthrough, clarification of tricky idioms, a traced example, noted bugs/smells, and improvement suggestions — pitched at someone new to the codebase.

43

DEBUGGING & CODE REVIEW

Perform a Senior-Level Code Review

WHAT IS THIS PROMPT FOR?

Use this to get a thorough, prioritized review of your code before opening a PR — covering correctness, readability, security, and performance, the way a thoughtful senior engineer would.

AI PROMPT — copy & paste

PROMPT

```
You are a principal engineer doing a rigorous but constructive code review.
Review the code below.

Code / diff:
[PASTE CODE OR DIFF]

Context: [WHAT IT'S SUPPOSED TO DO, LANGUAGE/FRAMEWORK]

Review across these dimensions and ORGANIZE findings by severity
(Blocker / Should-fix / Nice-to-have):
1. Correctness & edge cases (off-by-one, null/empty, error paths, async).
2. Security (injection, auth, secrets, unsafe input handling).
3. Readability & maintainability (naming, structure, duplication, comments).
4. Performance (obvious inefficiencies, N+1, unnecessary work).
5. Testing (what's untested, what tests to add).
6. Adherence to common best practices/conventions for this stack.

For each finding: explain WHY it matters and show the suggested change. End
with the top 3 things to fix first. Be direct but kind.
```

EXAMPLE RESULT

A severity-organized review (blockers first) covering correctness, security, readability, performance, and testing — each finding explained and shown as a concrete change, ending with a top-3 fix list.

44

DEBUGGING & CODE REVIEW

Find and Fix a Memory Leak

WHAT IS THIS PROMPT FOR?

Use this when your app's memory usage grows over time and eventually crashes or slows down. Common with event listeners, timers, subscriptions, and caches.

AI PROMPT — copy & paste

PROMPT

```
Act as a performance engineer specializing in memory issues. I suspect a
memory leak.

Symptoms: [E.G. "memory climbs steadily and never drops; eventually OOM"]
Environment: [NODE.JS / BROWSER / REACT, etc.]
Relevant code: [PASTE COMPONENTS/MODULES THAT MANAGE LISTENERS, TIMERS,
SUBSCRIPTIONS, CACHES, OR CLOSURES]

Help me by:
1. Listing the most common causes of leaks in this environment (uncleared
   intervals/timeouts, unremoved event listeners, lingering subscriptions,
   detached DOM nodes, unbounded caches, closures holding references).
2. Pinpointing the likely culprit(s) in my code and explaining the mechanism.
3. Providing the corrected code with proper cleanup (e.g. useEffect cleanup,
   removeEventListener, unsubscribe, bounded cache).
4. Explaining how to confirm the leak is gone using profiling tools (heap
   snapshots, what to look for).

Give me a short checklist to audit the rest of the codebase for the same
pattern.
```

EXAMPLE RESULT

A diagnosis of the leak mechanism, the specific culprit in your code, corrected code with proper cleanup, instructions to verify via heap snapshots, and a checklist to audit similar patterns.

45

DEBUGGING & CODE REVIEW

Debug an Async / Promise Issue

WHAT IS THIS PROMPT FOR?

Use this when async code misbehaves — unhandled rejections, code running out of order, awaits that don't wait, or data arriving as `undefined`. Async bugs are among the most confusing.

AI PROMPT — copy & paste

PROMPT

```
You are an expert in asynchronous JavaScript/TypeScript. My async code is
behaving incorrectly.

Expected behavior: [WHAT SHOULD HAPPEN]
Actual behavior: [WHAT HAPPENS — e.g. "result is undefined", "runs out of
order", "unhandled rejection"]
Code: [PASTE THE ASYNC CODE]

Diagnose and fix:
1. Identify the exact async mistake (missing await, not returning a promise,
   awaiting in a forEach, fire-and-forget, mixing callbacks/promises, racing
   state updates, unhandled rejection).
2. Explain the event-loop / promise behavior that causes the observed result.
3. Provide corrected code using proper async/await patterns (and
   Promise.all / for...of where parallel vs sequential matters).
4. Ensure errors are handled and nothing is silently swallowed.
5. Note how to make this kind of bug visible earlier (linting, error
   handling).

Be explicit about which parts should run in parallel vs sequence.
```

EXAMPLE RESULT

The precise async mistake identified, an event-loop-level explanation of the symptom, corrected code with proper await/parallelism and error handling, and tips to catch such bugs earlier.

46

DEBUGGING & CODE REVIEW

Fix a React Re-render or Infinite Loop Bug

WHAT IS THIS PROMPT FOR?

Use this when a component re-renders endlessly, a `useEffect` fires repeatedly, or state updates trigger a loop and crash the page — classic React footguns.

AI PROMPT — copy & paste

PROMPT

```
Act as a React expert. My component has an infinite render loop or excessive
re-renders.

Symptom: [E.G. "Maximum update depth exceeded", "useEffect runs forever",
"page freezes"]
Code: [PASTE THE COMPONENT AND ANY HOOKS/EFFECTS]

Diagnose and fix:
1. Identify the exact cause (setState during render, missing/incorrect
   useEffect dependency array, new object/array/function identity each render
   used as a dependency, state derived in a loop, etc.).
2. Explain why it loops, step by step.
3. Provide corrected code (proper dependency arrays, useMemo/useCallback only
   where needed, moving derived values, functional updates).
4. Confirm the fix won't introduce stale-closure bugs.
5. Give a rule of thumb to avoid this in future.

Explain the dependency-array reasoning clearly.
```

EXAMPLE RESULT

The exact cause of the loop, a step-by-step explanation, corrected code with proper dependencies and stable identities, a stale-closure check, and a rule of thumb to prevent recurrence.

47

DEBUGGING & CODE REVIEW

Diagnose and Fix a CORS Error

WHAT IS THIS PROMPT FOR?

Use this when your frontend can't talk to your backend because of CORS errors in the console. CORS confuses almost everyone — this prompt explains it and fixes it correctly (not by disabling security).

AI PROMPT — copy & paste

PROMPT

```
You are a web engineer who deeply understands CORS. I'm getting a CORS error.

Error in console: [PASTE EXACT ERROR]
Setup: frontend at [ORIGIN], backend at [ORIGIN], framework [E.G. Express],
request type [GET/POST...], using cookies/credentials? [YES/NO]
Relevant server/client code: [PASTE]

Explain and fix:
1. In plain language, explain what CORS is and what this specific error means.
2. Identify whether the problem is missing headers, the preflight (OPTIONS)
   request, credentials handling, or a misconfigured origin.
3. Provide the correct server-side fix (e.g. proper cors() config in Express)
   with an explicit origin allowlist — NOT a wildcard when credentials are
   used, and explain why.
4. If credentials/cookies are involved, show both the server (credentials:
   true) and client (withCredentials / credentials: "include") changes.
5. Note why disabling CORS or using "*" blindly is the wrong fix.

Confirm the secure, correct configuration.
```

EXAMPLE RESULT

A plain explanation of the specific CORS error, identification of the real cause (preflight, credentials, or origin), the correct server and client configuration with an origin allowlist, and why wildcards are unsafe.

48

DEBUGGING & CODE REVIEW

Debug a Failing Test

WHAT IS THIS PROMPT FOR?

Use this when a unit or integration test fails and the message isn't obvious — flaky tests, wrong assertions, bad mocks, or an actual bug the test correctly caught.

AI PROMPT — copy & paste

PROMPT

```
Act as a testing expert. A test is failing and I need to understand why.

Test framework: [JEST / VITEST / etc.]
Failing test code: [PASTE TEST]
Code under test: [PASTE IMPLEMENTATION]
Failure output: [PASTE THE ERROR/DIFF]

Determine:
1. Whether the test is correct and caught a real bug, OR the test itself is
   wrong (bad assertion, incorrect mock, wrong expected value, async not
   awaited, shared state between tests).
2. The exact reason for the failure, referencing the diff/output.
3. The fix — either correcting the implementation or correcting the test —
   and which one is appropriate here.
4. If the test is flaky, identify the source of nondeterminism (time,
   randomness, order dependence, network) and how to make it deterministic.
5. Whether the test is actually testing the right thing; suggest a better
   assertion if not.

Show the corrected code.
```

EXAMPLE RESULT

A clear verdict on whether the bug is in the code or the test, the exact failure reason tied to the output, the correct fix, flakiness sources if any, and a stronger assertion where warranted.

49

DEBUGGING & CODE REVIEW

Find and Fix a Race Condition

WHAT IS THIS PROMPT FOR?

Use this when bugs appear intermittently — duplicate records, inconsistent state, or "works on my machine" issues caused by concurrent operations happening in an unexpected order.

AI PROMPT — copy & paste

PROMPT

```
You are a concurrency expert. I think I have a race condition.

Symptom: [E.G. "sometimes two records are created", "balance goes negative
under load", "state is occasionally inconsistent"]
Environment: [NODE.JS / BROWSER / DB], relevant code: [PASTE]

Help me:
1. Confirm whether this is a race condition and explain the interleaving of
   operations that produces the bug (a concrete timeline of events).
2. Identify the unprotected shared resource or check-then-act sequence.
3. Provide a correct fix appropriate to the context: e.g. database
   transactions + proper isolation level, unique constraints, atomic
   operations, optimistic/pessimistic locking, idempotency keys, debouncing,
   or request deduplication.
4. Explain the trade-offs of the chosen approach (performance, contention).
5. Suggest how to reproduce/test it (simulating concurrency).

Be concrete about the locking or atomicity mechanism and why it closes the
window.
```

EXAMPLE RESULT

Confirmation of the race with a concrete event-interleaving timeline, the unprotected resource identified, a context-appropriate fix (transaction, constraint, lock, or idempotency key), trade-offs, and a way to reproduce it.

50

DEBUGGING & CODE REVIEW

Review Code for Security Vulnerabilities

WHAT IS THIS PROMPT FOR?

Use this to audit code for security holes before shipping — injection, auth flaws, exposed secrets, unsafe input handling. A focused security pass that general reviews often miss.

AI PROMPT — copy & paste

PROMPT

```
Act as an application security engineer doing a focused security audit.
Review the code below for vulnerabilities.

Code: [PASTE CODE — e.g. API routes, auth logic, DB queries, input handling]
Context: [WHAT IT DOES, STACK]

Audit for (at minimum):
1. Injection (SQL/NoSQL, command, etc.) and unsafe query construction.
2. Broken authentication/authorization (missing checks, IDOR, privilege
   escalation, trusting client input for identity/role).
3. Sensitive data exposure (secrets in code, leaking errors, logging PII,
   weak hashing).
4. Input validation/sanitization gaps and XSS where output is rendered.
5. Insecure configuration (CORS, cookies, headers) and dependency risks.

For each issue: rate severity (Critical/High/Medium/Low), explain the exploit
scenario, and show the secure fix. Map findings to OWASP Top 10 categories
where relevant. List the most critical fix first.
```

EXAMPLE RESULT

A severity-rated vulnerability report mapped to OWASP categories, each with an exploit scenario and a secure fix, ordered most-critical-first — a practical pre-ship security pass.

51

DEBUGGING & CODE REVIEW

Optimize a Slow Database Query

WHAT IS THIS PROMPT FOR?

Use this when an endpoint is slow and the database is the bottleneck — missing indexes, N+1 queries, or inefficient joins. The AI reads the query plan and tells you exactly what to fix.

AI PROMPT — copy & paste

PROMPT

```
You are a database performance expert. Help me optimize a slow query.

Database: [POSTGRESQL / MYSQL / MONGODB]
The query (or ORM call): [PASTE]
Relevant schema/indexes: [PASTE TABLE/COLLECTION DEFINITIONS + EXISTING
INDEXES]
EXPLAIN/EXPLAIN ANALYZE output (if available): [PASTE]
Approx row counts: [PER TABLE]

Optimize:
1. Read the query plan and identify the bottleneck (sequential scans,
   missing index, bad join order, sorting/temp files, N+1 from the ORM).
2. Recommend specific indexes (including composite/covering indexes) and
   explain why each helps and the write-cost trade-off.
3. Rewrite the query if structure is the problem (avoid SELECT *, fix joins,
   push filtering down, paginate).
4. For ORM N+1s, show how to eager-load/batch correctly.
5. Estimate the expected impact and how to verify with EXPLAIN ANALYZE.

Be specific about the exact index definitions to create.
```

EXAMPLE RESULT

A query-plan diagnosis, exact index definitions to add (with trade-offs), a rewritten query where needed, an ORM N+1 fix, and a way to verify the improvement with `EXPLAIN ANALYZE`.

52

DEBUGGING & CODE REVIEW

Decode a Confusing Error Message

WHAT IS THIS PROMPT FOR?

Use this when you hit a cryptic error you've never seen — from a library, build tool, or runtime — and need it translated into plain language with a path to a fix.

AI PROMPT — copy & paste

PROMPT

```
Act as an experienced engineer. Decode this confusing error message for me.

Error: [PASTE THE FULL ERROR, INCLUDING ANY CODES OR LIBRARY NAMES]
What I was doing when it happened: [CONTEXT]
Stack/tooling: [LANGUAGE, FRAMEWORK, BUILD TOOL, VERSIONS IF KNOWN]

Explain:
1. In plain English, what this error is actually saying.
2. The usual root causes for this specific error, ranked by likelihood.
3. The most likely fix given my context, with exact steps/commands.
4. How to verify the fix worked.
5. If it's a known library/tooling quirk, mention that and the typical
   resolution.

If the error alone isn't enough to be sure, tell me exactly what additional
info or files to share.
```

EXAMPLE RESULT

A plain-language translation of the error, ranked likely causes, the most probable fix with exact steps, a verification method, and a request for specifics if the message alone is ambiguous.

53

DEBUGGING & CODE REVIEW

Refactor Legacy Code Safely

WHAT IS THIS PROMPT FOR?

Use this when you need to improve messy, working code without breaking it. The AI proposes incremental refactors that preserve behavior — not a risky rewrite.

AI PROMPT — copy & paste

PROMPT

```
You are a refactoring expert who values behavior preservation above all.
Refactor the following legacy code to be cleaner and more maintainable
WITHOUT changing its external behavior.

Code: [PASTE]
Constraints: [E.G. "must keep the same public API/signature", "no new deps"]

Approach:
1. First summarize what the code currently does and any behavior I must
   preserve (including quirks/edge cases that callers may rely on).
2. Identify the specific problems (long functions, duplication, poor naming,
   tight coupling, deep nesting, mixed responsibilities).
3. Refactor in small, safe, ordered steps — describe each step and show the
   resulting code, so I can apply and verify them incrementally.
4. Recommend characterization tests to add BEFORE refactoring to lock in
   current behavior.
5. Confirm the public interface is unchanged.

Prioritize readability and low risk over cleverness.
```

EXAMPLE RESULT

A summary of current behavior to preserve, a list of concrete problems, an ordered set of small safe refactor steps with code, recommended characterization tests, and confirmation the public API is unchanged.

54

DEBUGGING & CODE REVIEW

Find the Root Cause (Root Cause Analysis)

WHAT IS THIS PROMPT FOR?

Use this when a bug keeps coming back or you've fixed symptoms without understanding the underlying cause. Drives toward the real source instead of patching surface issues.

AI PROMPT — copy & paste

PROMPT

```
Act as a senior engineer leading a root-cause analysis. Help me find the TRUE
cause of a recurring problem, not just the symptom.

Problem: [DESCRIBE THE BUG/INCIDENT AND HOW OFTEN IT RECURS]
What we've tried/observed: [LIST]
Relevant code/logs/config: [PASTE]

Lead me through:
1. A "5 Whys" style chain from the symptom down to the underlying cause —
   ask me targeted questions if a link is missing.
2. Separate the proximate cause (what directly triggered it) from the root
   cause (the deeper reason it was possible).
3. Identify whether the root cause is in code, design, process, or
   assumptions.
4. Propose a fix that addresses the ROOT cause so it can't recur, plus a
   quick mitigation if needed now.
5. Recommend a guardrail (test, type, validation, alert, process change) to
   prevent the whole class of issue.

Be rigorous; don't stop at the first plausible cause.
```

EXAMPLE RESULT

A 5-Whys chain from symptom to root cause, proximate-vs-root separation, a root-level fix plus quick mitigation, and a guardrail to prevent the entire class of problem — with targeted questions where info is missing.

55

DEBUGGING & CODE REVIEW

Debug a Production Incident

WHAT IS THIS PROMPT FOR?

Use this during or after a production outage when you need to triage fast, stop the bleeding, and then investigate calmly. Brings structure to a high-pressure moment.

AI PROMPT — copy & paste

PROMPT

```
You are an experienced on-call SRE. Help me triage and resolve a production
incident calmly and systematically.

What's happening: [SYMPTOMS, ERROR RATES, AFFECTED USERS/ENDPOINTS]
When it started + recent changes: [DEPLOYS, CONFIG, TRAFFIC SPIKES]
Available signals: [LOGS, METRICS, ALERTS — PASTE RELEVANT EXCERPTS]

Guide me through:
1. IMMEDIATE: how to assess blast radius and the fastest safe mitigation
   (rollback, feature flag, scale up, circuit-break) to stop user impact —
   before full diagnosis.
2. A prioritized list of likely causes given the timing and recent changes,
   and the single highest-signal thing to check first.
3. What to look for in logs/metrics to confirm or rule out each hypothesis.
4. Once mitigated, how to confirm recovery.
5. A short post-incident checklist (root cause, action items, prevention).

Prioritize stopping user impact over finding the perfect explanation.
```

EXAMPLE RESULT

An incident playbook: immediate blast-radius assessment and safe mitigation, ranked hypotheses tied to recent changes, the highest-signal check first, recovery confirmation, and a post-incident checklist.

56

DEBUGGING & CODE REVIEW

Review a Pull Request

WHAT IS THIS PROMPT FOR?

Use this when reviewing a teammate's PR (or your own before requesting review) and want a structured, fair assessment that catches issues without nitpicking trivia.

AI PROMPT — copy & paste

PROMPT

```
Act as a thoughtful senior reviewer. Review this pull request and give
feedback I can paste as review comments.

PR description / goal: [WHAT IT'S MEANT TO ACCOMPLISH]
The diff: [PASTE DIFF]

Provide:
1. A quick verdict: does the change accomplish its stated goal? Any scope
   creep or missing pieces?
2. Findings grouped as: must-fix (correctness/security/breaking), should-fix
   (maintainability/tests), and optional (style/nits — keep these minimal).
3. For each must/should item: the file/line area, why it matters, and a
   concrete suggested change.
4. Whether the tests adequately cover the change; what test cases are missing.
5. Anything that should be documented or that affects other parts of the
   system.
6. One or two genuine positives.

Be specific and constructive; avoid bikeshedding and subjective style debates.
```

EXAMPLE RESULT

A goal-check verdict, findings grouped by must/should/optional with concrete suggestions, a test-coverage assessment, documentation/impact notes, and genuine positives — ready to paste as PR comments.

57

DEBUGGING & CODE REVIEW

Identify and Fix Code Smells

WHAT IS THIS PROMPT FOR?

Use this when code "works" but feels wrong and you can't articulate why. The AI names the specific smells and shows the refactoring patterns that address each.

AI PROMPT — copy & paste

PROMPT

```
You are a clean-code expert. Identify code smells in the following code and
show me how to fix each one.

Code: [PASTE]

For your analysis:
1. Name each specific code smell you find (e.g. long method, large class,
   duplicated code, feature envy, primitive obsession, long parameter list,
   shotgun surgery, magic numbers, deep nesting, god object).
2. For each smell: explain why it's a problem in practical terms (what it
   makes harder).
3. Recommend the specific refactoring technique that addresses it (e.g.
   extract method, introduce parameter object, replace conditional with
   polymorphism, guard clauses).
4. Show the refactored code applying these techniques.
5. Prioritize: which smells matter most here and which are minor.

Don't over-engineer — only refactor where it genuinely improves the code.
```

EXAMPLE RESULT

A named list of code smells with practical explanations, the matching refactoring technique for each, refactored code, and a priority ranking — without over-engineering.

58

DEBUGGING & CODE REVIEW

Fix a Build or Compilation Error

WHAT IS THIS PROMPT FOR?

Use this when your project won't build — TypeScript errors, bundler failures, dependency conflicts, or config issues blocking you from even running the code.

AI PROMPT — copy & paste

PROMPT

```
Act as a build-tooling expert. My project won't build/compile. Help me fix it.

Build/compile error: [PASTE FULL OUTPUT]
Tooling: [E.G. TYPESCRIPT, VITE/WEBPACK, NODE VERSION, PACKAGE MANAGER]
Relevant config: [PASTE tsconfig / bundler config / package.json SNIPPETS]
What changed recently: [NEW DEP, VERSION BUMP, CONFIG EDIT, NOTHING]

Diagnose:
1. Parse the error and explain what the build tool is complaining about.
2. Identify the likely cause (type error, missing/mismatched dependency,
   version conflict, misconfigured path/alias, ESM vs CommonJS mismatch,
   missing types).
3. Give the exact fix — config change, command, or code edit — step by step.
4. If it's a dependency/version conflict, show how to inspect and resolve it
   (lockfile, peer deps).
5. How to verify the build succeeds and avoid the issue recurring.

Give exact commands and config diffs.
```

EXAMPLE RESULT

An interpretation of the build error, the likely cause (types, deps, ESM/CJS, config), exact step-by-step fixes with commands and config diffs, and verification steps.

59

DEBUGGING & CODE REVIEW

Reproduce an Intermittent Bug

WHAT IS THIS PROMPT FOR?

Use this for the worst kind of bug — one that happens "sometimes." You can't fix what you can't reproduce, so this prompt builds a strategy to make it consistent and observable.

AI PROMPT — copy & paste

PROMPT

```
You are a debugging strategist. I have an intermittent bug I can't reliably
reproduce. Help me make it reproducible and then fix it.

Bug: [DESCRIBE WHAT HAPPENS AND HOW OFTEN]
What seems to correlate with it (if anything): [TIMING, LOAD, SPECIFIC USER/
DATA, DEVICE, ORDER OF ACTIONS]
Relevant code/area: [PASTE OR DESCRIBE]

Help me:
1. Brainstorm the categories of intermittent causes to consider (race
   conditions, uninitialized/stale state, time/timezone, caching, network
   flakiness, environment differences, random data, order dependence).
2. Design a reproduction strategy: what to control/vary, added logging/
   instrumentation, and conditions to force the bug to appear consistently.
3. Suggest targeted logging or assertions to capture state at the moment of
   failure.
4. Once reproduced, outline how to isolate the cause via bisection.
5. Recommend a fix approach and a regression test that would have caught it.

Start by asking me the 2-3 most useful clarifying questions.
```

EXAMPLE RESULT

A categorized list of intermittent-bug causes, a concrete reproduction and instrumentation strategy, an isolation plan via bisection, a fix approach, a regression test, and a few sharp clarifying questions.

60

DEBUGGING & CODE REVIEW

Compare Two Approaches (Trade-off Analysis)

WHAT IS THIS PROMPT FOR?

Use this when you're stuck between two technical choices — libraries, patterns, architectures — and want an objective comparison rather than a gut decision.

AI PROMPT — copy & paste

PROMPT

```
Act as a pragmatic staff engineer. Help me choose between two approaches with
a clear, unbiased trade-off analysis.

Decision: [E.G. "REST vs GraphQL", "Redux vs React Query", "monolith vs
services", "SQL vs NoSQL for this feature"]
My context: [APP, TEAM SIZE/EXPERIENCE, SCALE, TIMELINE, EXISTING STACK,
CONSTRAINTS]

Deliver:
1. A side-by-side comparison across the dimensions that matter HERE (dev
   speed, performance, scalability, complexity, maintainability, ecosystem,
   team familiarity, cost).
2. The concrete trade-offs of each — what I gain and give up.
3. Which scenarios favor option A vs option B.
4. A clear recommendation FOR MY SPECIFIC CONTEXT, with the reasoning, and
   what would change your recommendation.
5. The cost of being wrong / how reversible each choice is.

Don't sit on the fence — give a recommendation, but show your reasoning so I
can disagree.
```

EXAMPLE RESULT

A context-specific side-by-side comparison, the concrete trade-offs of each option, scenarios favoring each, a clear reasoned recommendation, and an honest note on reversibility and what would change the call.

PART 04

4

UI/UX & Design

20 prompts to design interfaces that are usable, accessible, and beautiful — even if you're not a designer.

Prompts 61 - 80

20 prompts

61 Create a Design System with Design Tokens 70

62 Improve the UX of an Existing Page 71

63 Design a Color Palette 72

64 Conduct a Heuristic Usability Audit 73

65 Design a User Onboarding Flow 74

66 Write UX Microcopy 75

67 Design an Effective Empty State 76

68 Improve Form UX 77

69 Run an Accessibility (WCAG) Audit 78

70 Design a Mobile-First Responsive Layout 79

71 Design a High-Converting Pricing Page 80

72 Create Wireframes from Requirements 81

73 Design Error and Success States 82

74 Choose a Typography Pairing 83

75 Design a Dashboard / Data Visualization Layout 84

76 Improve Visual Hierarchy 85

77 Design a Modal / Dialog Pattern 86

78 Create a Component Spec for Developers 87

79 Design a Dark Mode Variant 88

80 Optimize CTAs and Conversion Copy 89

61

UI/UX & DESIGN

Create a Design System with Design Tokens

WHAT IS THIS PROMPT FOR?

Use this when your UI looks inconsistent across pages and you want a single source of truth for colors, spacing, and typography. Essential for any project that will grow beyond a few screens.

AI PROMPT — copy & paste

PROMPT

```
You are a design systems engineer. Create a foundational design system as
design tokens for a [WEB APP / MARKETING SITE], implemented for Tailwind CSS.

Brand context: [INDUSTRY, VIBE — e.g. "fintech, trustworthy, modern" — and
any existing brand colors]

Deliver:
1. A color system: a primary scale (50–900), neutral/gray scale, and
   semantic colors (success, warning, error, info), each meeting WCAG AA
   contrast for text. Note which shades to use for backgrounds vs text.
2. A typographic scale (font sizes + line heights + weights) and a font
   pairing recommendation.
3. A spacing scale, border-radius scale, and shadow/elevation scale.
4. All of the above expressed as a tailwind.config theme extension AND as CSS
   variables (so they work outside Tailwind too).
5. Brief usage guidelines (when to use each semantic color, spacing rhythm).

Keep it practical and not bloated — a real working foundation.
```

EXAMPLE RESULT

A practical token set — color scales with semantic roles, type scale, spacing/radius/shadow scales — delivered as a Tailwind theme extension and CSS variables, plus concise usage guidelines.

62

UI/UX & DESIGN

Improve the UX of an Existing Page

WHAT IS THIS PROMPT FOR?

Use this when a page works but feels clunky and you want concrete, prioritized UX improvements rather than a vague "make it better."

AI PROMPT — copy & paste

PROMPT

```
Act as a senior UX designer doing a practical critique. Improve the user
experience of this page.

Page purpose & primary user goal: [WHAT USERS COME HERE TO DO]
Current page: [DESCRIBE THE LAYOUT/FLOW OR PASTE THE MARKUP/SCREENSHOT]
Known problems / metrics: [E.G. "high drop-off", "users miss the CTA"]

Critique and improve:
1. Evaluate whether the page makes the primary action obvious and easy.
2. Identify specific UX issues: unclear hierarchy, cognitive load, weak
   affordances, poor feedback, confusing copy, friction in the flow.
3. Give prioritized, concrete recommendations (high/medium/low impact) — for
   each, state the problem, the change, and the expected effect.
4. Address mobile experience and accessibility issues too.
5. Suggest what to A/B test if the change is uncertain.

Be specific ("move X above the fold and make it the only primary button"),
not generic ("improve usability").
```

EXAMPLE RESULT

A focused UX critique with prioritized, concrete recommendations — each naming the problem, the specific change, and the expected effect — covering hierarchy, friction, mobile, accessibility, and what to test.

63

UI/UX & DESIGN

Design a Color Palette

WHAT IS THIS PROMPT FOR?

Use this when starting a project and you need a cohesive, accessible color palette but aren't confident picking colors. Produces something professional you can use immediately.

AI PROMPT — copy & paste

PROMPT

```
You are a color and brand expert. Create a cohesive, accessible color palette
for a [PROJECT TYPE] with this personality: [E.G. "calm, premium, trustworthy"
/ "energetic, playful, bold"].

Deliver:
1. A primary brand color and the reasoning (color psychology fit for the
   brand and audience).
2. A full palette: primary, secondary/accent, neutrals, and semantic colors
   (success/warning/error/info) — give exact HEX values.
3. For each, provide a usable tint/shade scale (e.g. 50-900).
4. Accessibility: verify text/background combinations meet WCAG AA (4.5:1
   for body text) and list which pairings are safe for text vs decoration.
5. Example UI usage: background, surface, text, primary button, link, border.
6. A dark-mode variant of the palette.

Present it as a clean spec with HEX codes I can paste into Tailwind/CSS.
```

EXAMPLE RESULT

A cohesive palette with reasoning, full tint/shade scales in HEX, WCAG-checked text pairings, semantic colors, concrete UI usage mapping, and a dark-mode variant — ready to paste into your config.

64

UI/UX & DESIGN

Conduct a Heuristic Usability Audit

WHAT IS THIS PROMPT FOR?

Use this to evaluate an interface against established usability principles (Nielsen's heuristics) and get a structured list of issues — like a professional UX audit without hiring one.

AI PROMPT — copy & paste

PROMPT

```
Act as a UX researcher conducting a heuristic evaluation using Nielsen's 10
usability heuristics. Audit the following interface.

Interface: [DESCRIBE THE SCREEN/FLOW OR PASTE MARKUP/SCREENSHOT]
Primary tasks users perform: [LIST]

For the audit:
1. Go through each relevant heuristic (visibility of system status, match to
   real world, user control/freedom, consistency, error prevention,
   recognition over recall, flexibility, aesthetic/minimalist design, error
   recovery, help/docs).
2. For each, note whether the interface passes or has issues; describe each
   issue concretely.
3. Rate each issue's severity (0–4: cosmetic → catastrophic).
4. Give a specific recommendation to fix each issue.
5. Summarize the top 5 issues to address first by severity × frequency.

Be concrete and reference the actual elements, not abstractions.
```

EXAMPLE RESULT

A heuristic-by-heuristic audit with concrete issues, severity ratings (0–4), specific fixes, and a prioritized top-5 list — a structured, professional-style usability evaluation.

65

UI/UX & DESIGN

Design a User Onboarding Flow

WHAT IS THIS PROMPT FOR?

Use this when new users sign up but don't reach the "aha" moment. The AI designs an onboarding flow that gets users to value quickly and reduces early churn.

AI PROMPT — copy & paste

PROMPT

```
You are a product designer specializing in activation and onboarding. Design
an onboarding flow for my product.

Product: [WHAT IT IS]
The "aha moment" / core value: [WHAT MAKES USERS GET IT]
Target user: [WHO], signup context: [WEB APP / MOBILE]

Design:
1. Map the ideal path from signup to the aha moment in the fewest steps
   (define the activation milestone explicitly).
2. Decide the onboarding pattern(s) that fit: progressive disclosure, a setup
   checklist, contextual tooltips, an interactive product tour, sample/seed
   data, or empty-state guidance — and justify the choice.
3. Step-by-step flow with what each screen asks/shows and why; defer anything
   non-essential.
4. Reduce friction: what to remove, what to delay, sensible defaults, and
   personalization based on a single key question.
5. Re-engagement: what to do if a user drops off mid-onboarding.
6. Metrics to track activation success.

Keep it short — the best onboarding gets out of the way fast.
```

EXAMPLE RESULT

An onboarding design: a defined activation milestone, the right onboarding pattern with justification, a lean step-by-step flow, friction-reduction tactics, drop-off re-engagement, and activation metrics.

66

UI/UX & DESIGN

Write UX Microcopy

WHAT IS THIS PROMPT FOR?

Use this when buttons, error messages, tooltips, and empty states need clear, on-brand wording. Good microcopy quietly improves usability and trust everywhere.

AI PROMPT — copy & paste

PROMPT

```
Act as a UX writer. Write clear, helpful microcopy for the following UI
elements, matching this brand voice: [E.G. "friendly but professional",
"concise and confident"].

Context: [WHAT THE PRODUCT DOES, WHO THE USER IS]
Elements I need copy for:
[LIST — e.g. primary CTA buttons, an empty state for "no projects yet", a
form validation error for invalid email, a delete-confirmation dialog, a
success toast, a loading message, tooltip for a tricky setting]

For each element:
1. Provide the recommended copy (and 1-2 alternatives where tone could vary).
2. Keep it concise, action-oriented, and human — avoid jargon and blame
   ("Please enter a valid email" → friendlier, clearer).
3. For errors: say what happened and how to fix it, never just "error".
4. For confirmations of destructive actions: be explicit about consequences.
5. Ensure consistency in terminology and capitalization across all of them.

Present as a table: element → copy → short rationale.
```

EXAMPLE RESULT

A table of polished microcopy for each element with alternatives and rationale — action-oriented buttons, helpful errors, clear destructive confirmations, and consistent terminology in your brand voice.

67

UI/UX & DESIGN

Design an Effective Empty State

WHAT IS THIS PROMPT FOR?

Use this for the first thing users see before they have any data — empty dashboards, lists, or inboxes. A good empty state guides users to act instead of leaving them confused.

AI PROMPT — copy & paste

PROMPT

```
You are a product designer. Design an effective empty state for [E.G. "a
projects list when the user has no projects yet"].

Context: [WHAT THE FEATURE DOES, WHAT THE USER SHOULD DO FIRST]

Design the empty state covering:
1. The purpose: orient the user, reduce confusion, and prompt the right first
   action.
2. Content: a clear headline, a one-line explanation of the value, a primary
   CTA (the single most useful next action), and an optional secondary action
   or link to docs/sample data.
3. Visual treatment guidance (illustration/icon, not just blank space) and
   layout.
4. Differentiate the THREE empty-state types and tell me which applies:
   first-use (no data yet), user-cleared (e.g. all tasks done — celebrate),
   and no-results (search/filter returned nothing — offer to adjust).
5. Provide the actual copy and a React + Tailwind component implementing it.

Make it encouraging and action-oriented, never a dead end.
```

EXAMPLE RESULT

A purposeful empty-state design with headline, value statement, primary CTA, visual guidance, the correct empty-state type identified, real copy, and a React + Tailwind component implementing it.

68

UI/UX & DESIGN

Improve Form UX

WHAT IS THIS PROMPT FOR?

Use this when a form has high abandonment or frustrates users. Forms are where conversions die — this prompt applies proven form-design best practices.

AI PROMPT — copy & paste

PROMPT

```
Act as a UX designer specializing in forms and conversion. Improve the UX of
this form.

Form purpose: [E.G. SIGNUP / CHECKOUT / SETTINGS]
Current fields & layout: [LIST FIELDS, DESCRIBE LAYOUT, OR PASTE MARKUP]
Problems: [E.G. "high abandonment", "many validation errors"]

Improve it by addressing:
1. Field reduction: which fields are unnecessary or can be deferred/derived;
   ask only for what's truly needed now.
2. Layout: single-column, logical grouping, label placement, and field order.
3. Input design: correct input types/keyboards, autocomplete attributes,
   smart defaults, masks/formatting, and clear required vs optional marking.
4. Validation: inline, real-time-but-not-annoying, validate on blur, helpful
   error messages, and never clear the user's input on error.
5. Feedback: clear progress (if multi-step), disabled-until-valid submit,
   and a visible submitting/success state.
6. Accessibility: labels, focus order, error association (aria-describedby).

Give prioritized recommendations and a revised field list/layout.
```

EXAMPLE RESULT

Prioritized form improvements — fewer fields, single-column layout, correct input types and autocomplete, friendly inline validation, clear feedback, and accessibility fixes — plus a revised field list.

69

UI/UX & DESIGN

Run an Accessibility (WCAG) Audit

WHAT IS THIS PROMPT FOR?

Use this to check whether your UI is usable by people with disabilities and meets WCAG standards — a legal requirement in many places and simply the right thing to do.

AI PROMPT — copy & paste

PROMPT

```
You are an accessibility (a11y) specialist. Audit the following UI against
WCAG 2.1 AA and tell me how to fix the issues.

Component/page markup: [PASTE HTML/JSX]
Context: [WHAT IT DOES, ANY INTERACTIONS]

Audit for:
1. Semantic structure: correct landmarks, heading order, lists, and use of
   semantic elements vs divs.
2. Keyboard accessibility: everything operable without a mouse, logical focus
   order, visible focus indicators, no keyboard traps.
3. ARIA: correct roles/states/properties — and where ARIA is misused or
   unnecessary (prefer native elements).
4. Forms: labels, error association, instructions.
5. Color & contrast: text contrast ratios, not relying on color alone.
6. Images/media: alt text, decorative vs informative.
7. Dynamic content: focus management for modals/menus, live regions for
   updates.

For each issue: cite the relevant WCAG criterion, rate severity, and show the
corrected markup. End with a manual + automated testing checklist.
```

EXAMPLE RESULT

A WCAG-AA audit citing specific criteria, severity-rated issues across semantics/keyboard/ARIA/contrast/media, corrected markup for each, and a combined manual-plus-automated testing checklist.

70

UI/UX & DESIGN

Design a Mobile-First Responsive Layout

WHAT IS THIS PROMPT FOR?

Use this when you need a layout that works beautifully on phones first, then scales up — instead of cramming a desktop design onto small screens.

AI PROMPT — copy & paste

PROMPT

```
Act as a responsive design expert. Design a mobile-first responsive layout
for [DESCRIBE THE PAGE — e.g. "a product detail page" / "a dashboard"].

Content/sections to include: [LIST]

Provide:
1. The mobile (smallest) layout first: stacking order, what's prioritized
   above the fold, and what's hidden behind taps/accordions.
2. How the layout adapts at tablet and desktop breakpoints (when columns
   appear, when nav changes, when the sidebar shows) — define the breakpoints.
3. Touch considerations: tap target sizes (≥ 44px), spacing, thumb-reachable
   primary actions, no hover-only interactions.
4. Performance: image handling and avoiding layout shift across breakpoints.
5. The implementation as React + Tailwind using mobile-first utility classes
   (base styles for mobile, sm:/md:/lg: for larger).

Show the responsive component and annotate the breakpoint decisions.
```

EXAMPLE RESULT

A mobile-first layout plan with prioritized stacking, defined breakpoints and how the layout adapts, touch-target and performance considerations, and an annotated React + Tailwind implementation.

71

UI/UX & DESIGN

Design a High-Converting Pricing Page

WHAT IS THIS PROMPT FOR?

Use this when building or improving a SaaS pricing page — one of the highest-stakes pages on any product site. Combines UX, psychology, and clear plan presentation.

AI PROMPT — copy & paste

PROMPT

```
You are a product designer and conversion specialist. Design a pricing page
for my SaaS.

Product: [WHAT IT IS], plans: [LIST PLANS, PRICES, KEY FEATURES/LIMITS],
target customer: [WHO]

Design covering:
1. Plan structure and layout: how many tiers to show, ordering, and which
   plan to visually highlight as "recommended" and why.
2. Feature presentation: a clear comparison (cards and/or a comparison table)
   that makes differences scannable; lead with value, not feature dumps.
3. Pricing psychology applied honestly: anchoring, monthly/annual toggle with
   savings shown, framing of the recommended tier.
4. Trust & friction reducers: FAQ, money-back/cancel-anytime, social proof
   placement, and a clear CTA per plan.
5. Handling edge cases: a free/enterprise tier, "contact us", usage limits.
6. Accessibility and mobile layout (how the comparison table behaves on
   phones).
7. The React + Tailwind implementation for the pricing cards + toggle.

Give the layout rationale, then the code.
```

EXAMPLE RESULT

A pricing-page design: tier structure with a justified highlighted plan, scannable comparison, honest pricing psychology, trust elements, mobile handling, and a React + Tailwind implementation of the cards and billing toggle.

72

UI/UX & DESIGN

Create Wireframes from Requirements

WHAT IS THIS PROMPT FOR?

Use this when you have a feature idea or requirements but no design, and need a low-fidelity structural blueprint before writing code or hiring a designer.

AI PROMPT — copy & paste

PROMPT

```
Act as a UX designer. Turn the following requirements into low-fidelity
wireframe descriptions I can build or hand to a designer.

Feature/requirements: [DESCRIBE WHAT THE FEATURE MUST DO AND USER GOALS]
Platform: [WEB / MOBILE], constraints: [ANY]

For each screen/state in the flow:
1. Describe the layout structurally (regions, sections, and their hierarchy)
   using a text wireframe / ASCII-style block layout.
2. List the key components on each screen and their purpose (no visual
   styling — structure only).
3. Define the primary action and where it sits.
4. Note the important states: loading, empty, error, success.
5. Describe the flow/navigation between screens.
6. Call out the most important content to prioritize visually.

Keep it low-fidelity and focused on structure and flow, not aesthetics. Cover
the full happy path plus key edge states.
```

EXAMPLE RESULT

Structural, low-fidelity wireframe descriptions for each screen and state — regions, components, primary actions, navigation, and edge states — focused on layout and flow rather than visuals.

73

UI/UX & DESIGN

Design Error and Success States

WHAT IS THIS PROMPT FOR?

Use this when your app handles the happy path but feels broken when things go wrong (or right). Well-designed states make an app feel trustworthy and polished.

AI PROMPT — copy & paste

PROMPT

```
You are a UX designer. Design the error and success states for [DESCRIBE THE
FEATURE/FLOW — e.g. "submitting a payment form"].

For this flow, design:
1. SUCCESS state: confirmation messaging, what the user sees next, clear next
   action, and whether to celebrate or stay subtle.
2. ERROR states — handle each distinctly:
   - Validation errors (user can fix): inline, specific, blame-free.
   - Recoverable system errors (e.g. network): explain + offer retry,
     preserve the user's input.
   - Unrecoverable errors: clear explanation + a path forward (contact/
     support/alternative).
3. Loading/in-progress state so users aren't left guessing.
4. Tone of all messaging (helpful, not technical jargon, never blaming the
   user).
5. Accessibility: announce errors/success to screen readers (live regions),
   move focus appropriately.
6. Provide the actual copy and a React + Tailwind example showing the states.

Make failure feel handled and recoverable, not scary.
```

EXAMPLE RESULT

A full set of state designs — success, distinct error types (validation/recoverable/unrecoverable), and loading — with blame-free copy, accessibility handling, and a React + Tailwind example.

74

UI/UX & DESIGN

Choose a Typography Pairing

WHAT IS THIS PROMPT FOR?

Use this when text looks amateurish and you need a professional font pairing and type scale, without design training. Typography is the fastest way to make a UI look credible.

AI PROMPT — copy & paste

PROMPT

```
Act as a typography expert. Recommend a typography system for a [PROJECT TYPE]
with this personality: [E.G. "modern and trustworthy" / "editorial and
elegant" / "technical and clean"].

Deliver:
1. A font pairing recommendation (heading + body) using freely available
   fonts (Google Fonts / system fonts). Give 2 pairing options and explain
   why each fits the brand. Avoid overused/cliché pairings unless justified.
2. When a single well-chosen typeface (with weights) is better than a pairing.
3. A modular type scale (e.g. base 16px with a ratio): exact sizes, line
   heights, and weights for h1–h6, body, small, and caption.
4. Guidance on line length (measure), paragraph spacing, and letter-spacing
   for headings.
5. The implementation as Tailwind config + the font import/link.
6. Accessibility notes (minimum sizes, contrast, not relying on light weights
   for body text).

Present a clean, copy-pasteable spec.
```

EXAMPLE RESULT

Two justified font-pairing options, guidance on single-typeface use, a complete modular type scale with sizes/line-heights/weights, readability guidance, a Tailwind implementation, and accessibility notes.

75

UI/UX & DESIGN

Design a Dashboard / Data Visualization Layout

WHAT IS THIS PROMPT FOR?

Use this when you have metrics to display and need to organize them into a clear, scannable dashboard — choosing the right charts and hierarchy instead of dumping numbers on a screen.

AI PROMPT — copy & paste

PROMPT

```
You are a data visualization and dashboard design expert. Help me design a
dashboard.

Audience & goal: [WHO USES IT AND WHAT DECISIONS THEY MAKE]
Metrics/data available: [LIST THE KEY METRICS AND THEIR TYPES]

Design:
1. Prioritize: which metrics matter most and belong at the top; apply the
   inverted-pyramid (most important, glanceable info first).
2. For each metric, recommend the RIGHT visualization (single stat/KPI,
   line for trends, bar for comparison, etc.) and why — avoid chart types
   that mislead.
3. Layout & grouping: logical sections, an F/Z reading pattern, and how the
   grid reflows on smaller screens.
4. Reduce clutter: what to remove, sensible use of color (and not relying on
   color alone), clear labels and units, and meaningful comparisons (vs prior
   period / target).
5. Interaction: filters, date ranges, drill-downs — only where they add value.
6. A React + Tailwind layout skeleton with placeholders noting where to wire
   in a charting library.

Prioritize clarity and decision-making over visual flashiness.
```

EXAMPLE RESULT

A dashboard design that prioritizes key metrics, matches each to the right chart type, groups and reflows them sensibly, reduces clutter, adds only useful interactions, and includes a React + Tailwind layout skeleton.

76

UI/UX & DESIGN

Improve Visual Hierarchy

WHAT IS THIS PROMPT FOR?

Use this when a screen feels flat or cluttered and users don't know where to look. Strong visual hierarchy guides the eye to what matters most.

AI PROMPT — copy & paste

PROMPT

```
Act as a visual designer. Improve the visual hierarchy of this UI so users
instantly know where to look and what's most important.

UI: [DESCRIBE OR PASTE MARKUP/SCREENSHOT]
The single most important thing on this screen: [WHAT THE USER SHOULD NOTICE/
DO FIRST]

Analyze and improve hierarchy using:
1. Size & scale: is the most important element appropriately dominant?
2. Color & contrast: drawing attention to primary actions, de-emphasizing
   secondary ones.
3. Spacing & grouping: using whitespace and proximity to create clear
   sections and relationships (Gestalt principles).
4. Typography: weight and size to distinguish headings, body, and labels.
5. Position & flow: aligning to the natural reading pattern.
6. Reducing competing elements: too many bold/colored/primary items dilute
   hierarchy — recommend what to tone down.

Give specific before→after changes for each element, and the reasoning.
Limit the screen to ONE primary action visually.
```

EXAMPLE RESULT

A hierarchy critique with specific before→after changes across size, color, spacing, typography, and position — ensuring one clearly dominant primary action and a screen that guides the eye.

77

UI/UX & DESIGN

Design a Modal / Dialog Pattern

WHAT IS THIS PROMPT FOR?

Use this when you need modals done right — accessible, non-intrusive, and used only when appropriate. Modals are frequently misused and inaccessible.

AI PROMPT — copy & paste

PROMPT

```
You are a UX engineer. Help me design and implement a modal/dialog correctly.

Use case: [E.G. "confirm deletion" / "edit profile in a dialog" / "show
details"]

Cover:
1. First, tell me whether a modal is even the right pattern here, or whether
   an inline expansion, a separate page, or a non-blocking panel would be
   better — and why.
2. If a modal fits, design it: clear title, focused content, primary +
   secondary actions, and a sensible size; avoid modals-on-modals.
3. Accessibility (critical): role="dialog"/aria-modal, focus moves into the
   modal on open and is TRAPPED, Escape closes, focus returns to the trigger
   on close, background is inert and non-scrollable, and the title is the
   accessible name.
4. Behavior: close on backdrop click (but not for destructive confirms),
   prevent body scroll, animate in/out.
5. For destructive actions: require explicit confirmation and label the
   button with the actual action ("Delete project", not "OK").
6. A reusable, accessible React + Tailwind <Modal> implementation.

Lead with the "should this be a modal?" decision.
```

EXAMPLE RESULT

A judgment on whether a modal is appropriate, a correct design if so, rigorous accessibility (focus trap, ESC, inert background, focus return), sensible behavior, destructive-action handling, and a reusable React + Tailwind modal.

78

UI/UX & DESIGN

Create a Component Spec for Developers

WHAT IS THIS PROMPT FOR?

Use this to translate a UI component idea into a precise spec a developer can build from without guessing — covering props, states, behavior, and edge cases.

AI PROMPT — copy & paste

PROMPT

```
Act as a senior product designer writing a component specification for an
engineer. Spec out the following component so it can be built with zero
ambiguity.

Component: [E.G. "a searchable multi-select dropdown"]
Where it's used / purpose: [CONTEXT]

Write a complete spec covering:
1. Purpose and usage guidelines (when to use / not use it).
2. Anatomy: the visual parts of the component.
3. Props/API: every configurable prop with type, default, and description.
4. ALL states: default, hover, focus, active, disabled, loading, error,
   empty, and selected — describe each.
5. Behavior & interactions: keyboard support, what each action does, edge
   cases (long text, many items, no results).
6. Accessibility requirements (roles, ARIA, keyboard, focus).
7. Responsive behavior.
8. Content guidelines (placeholder text, truncation rules).

Present it as a structured spec doc. Be exhaustive about states and edge
cases — that's where build ambiguity hides.
```

EXAMPLE RESULT

A precise, build-ready component spec: purpose, anatomy, a full props/API table, every interaction state, keyboard/accessibility requirements, responsive behavior, and content rules — leaving no ambiguity.

79

UI/UX & DESIGN

Design a Dark Mode Variant

WHAT IS THIS PROMPT FOR?

Use this when you have a light theme and need a proper dark mode — not just inverted colors, but a thoughtfully designed dark palette that's comfortable and accessible.

AI PROMPT — copy & paste

PROMPT

```
You are a UI designer experienced in dark mode. Design a dark-mode variant of
my existing light theme.

Current light palette: [PASTE COLORS — backgrounds, surfaces, text, primary,
borders]

Design the dark theme covering:
1. The core principle: dark mode is NOT just inverted colors. Use dark gray
   (not pure black) backgrounds, and lighten/desaturate brand colors so they
   don't vibrate on dark surfaces.
2. An elevation system: lighter surfaces = higher elevation (since shadows
   are weak on dark) — define background, surface, and elevated-surface
   colors.
3. Text colors: avoid pure white for body text (use slightly off-white) to
   reduce eye strain; define primary/secondary/disabled text.
4. Adjust accent/semantic colors for sufficient contrast on dark backgrounds
   (verify WCAG AA).
5. Borders, dividers, and how to handle images/illustrations in dark mode.
6. Deliver the dark palette as exact HEX values mapped to the same token
   names as the light theme, ready for a Tailwind dark: setup.

Explain each adjustment briefly.
```

EXAMPLE RESULT

A properly designed dark palette — dark-gray surfaces, an elevation system, off-white text, contrast-adjusted accents, and border/image handling — mapped to your token names as HEX, with brief rationale.

80

UI/UX & DESIGN

Optimize CTAs and Conversion Copy

WHAT IS THIS PROMPT FOR?

Use this when a page gets traffic but not conversions. The AI sharpens calls-to-action and key copy to be clearer and more compelling — without resorting to dark patterns.

AI PROMPT — copy & paste

PROMPT

```
Act as a conversion copywriter and UX strategist. Optimize the calls-to-
action and key copy on my page to increase conversions — ethically, no dark
patterns.

Page goal & desired action: [WHAT YOU WANT USERS TO DO]
Audience: [WHO], current copy: [PASTE HEADLINE, SUBHEAD, CTA BUTTONS, KEY
SECTIONS]

Improve:
1. The headline: make it benefit-led and specific (what the user gets), not
   feature- or company-centric. Give 3 options.
2. The subhead: support the headline and address the main objection.
3. CTA buttons: replace vague labels ("Submit", "Learn more") with specific,
   value-driven, first-person-friendly text; give options and explain.
4. Reduce friction & objections: what doubt is stopping users, and the copy/
   microcopy that addresses it (risk reversal, social proof placement).
5. Clarity over cleverness: cut jargon and fluff; tighten everything.
6. Suggest 1-2 things to A/B test.

Keep it honest and clear — persuasion through clarity and real value, not
manipulation. Present as before → after with reasoning.
```

EXAMPLE RESULT

Sharpened, benefit-led headline options, a supporting subhead, specific value-driven CTA labels, objection-addressing microcopy, and A/B test ideas — presented as before→after with reasoning, free of dark patterns.

PART 05

5

Productivity & Career

20 prompts to ship faster, document better, and grow your career.

Prompts 81 - 100

20 prompts

81 Generate a Professional README 91

82 Write Conventional Git Commit Messages 92

83 Document a Codebase 93

84 Write Code Comments and JSDoc 94

85 Write a Dockerfile and docker-compose Setup 95

86 Set Up a CI/CD Pipeline with GitHub Actions 96

87 Write a Technical Blog Post 97

88 Prepare for a Technical Interview 98

89 Optimize Your Developer Resume 99

90 Write a Test Suite 100

91 Write an Architecture Decision Record (ADR) 101

92 Break Down a Project into Tasks 102

93 Create a Study Plan for a New Technology 103

94 Write API Documentation (OpenAPI) 104

95 Estimate Project Time Realistically 105

96 Refactor Toward Clean Architecture 106

97 Write a Tailored Job Application 107

98 Build Your Developer Personal Brand 108

99 Do a Self Code-Review Before Opening a PR 109

100 Create a Deployment Checklist 110

81

PRODUCTIVITY & CAREER

Generate a Professional README

WHAT IS THIS PROMPT FOR?

Use this when your project's README is missing or weak. A great README is the first thing employers, clients, and contributors see — it makes or breaks first impressions on GitHub.

AI PROMPT — copy & paste

PROMPT

```
Act as an open-source maintainer. Write a professional, polished README.md
for my project.

Project details:
- Name & one-line description: [...]
- What problem it solves / key features: [...]
- Tech stack: [...]
- How to install & run locally: [STEPS / COMMANDS]
- Environment variables needed: [LIST]
- How to use it (basic example): [...]
- License: [...], repo URL: [...]

Produce a complete README in Markdown with:
1. A title, concise description, and (placeholder) badges (build, license,
   version).
2. A short "Features" list and, if useful, a screenshot/demo placeholder.
3. Tech stack, prerequisites, and clear step-by-step installation & setup.
4. Usage examples with code blocks.
5. Environment variable documentation (as a table).
6. Project structure overview, scripts, and a Contributing + License section.
7. A table of contents if the README is long.

Make it scannable, well-formatted, and welcoming to newcomers. Fill what you
can and clearly mark placeholders for what I must complete.
```

EXAMPLE RESULT

A complete, well-structured README in Markdown — description, badges, features, setup steps, usage examples, env-var table, project structure, and contributing/license sections — scannable and professional.

82

PRODUCTIVITY & CAREER

Write Conventional Git Commit Messages

WHAT IS THIS PROMPT FOR?

Use this when your commit history is a mess of "fix", "update", "wip". Clean, conventional commits make history readable and enable automated changelogs and versioning.

AI PROMPT — copy & paste

PROMPT

```
You are a Git workflow expert. Write clean commit messages following the
Conventional Commits specification for the changes I describe.

My changes: [DESCRIBE WHAT YOU DID, OR PASTE THE DIFF / git status]

For each logical change, produce a commit message that:
1. Uses the correct type (feat, fix, docs, style, refactor, perf, test,
   build, ci, chore) and an optional scope.
2. Has a concise, imperative-mood subject line under ~50 chars ("add" not
   "added").
3. Includes a body (when the change isn't trivial) explaining WHAT and WHY,
   not how.
4. Notes BREAKING CHANGE in the footer when applicable.
5. References issues if I mention any.

Also:
- If my described changes mix unrelated things, recommend how to SPLIT them
  into separate atomic commits, and give a message for each.
- Show the final message(s) in copy-paste-ready format.
```

EXAMPLE RESULT

Properly formatted Conventional Commit messages with correct types/scopes, imperative subjects, explanatory bodies, and breaking-change footers — plus advice on splitting mixed changes into atomic commits.

83

PRODUCTIVITY & CAREER

Document a Codebase

WHAT IS THIS PROMPT FOR?

Use this when a project lacks documentation and new developers (or future-you) struggle to understand it. Produces the architectural docs that READMEs alone don't cover.

AI PROMPT — copy & paste

PROMPT

```
Act as a senior engineer who writes excellent technical documentation. Help
me document my codebase so a new developer can become productive quickly.

Project overview: [WHAT IT IS, STACK]
Structure / key modules: [PASTE FOLDER TREE OR DESCRIBE MAIN PARTS]
Key flows: [E.G. AUTH FLOW, REQUEST LIFECYCLE]

Produce:
1. An ARCHITECTURE.md: high-level overview, the main components and how they
   interact, the request/data flow, and key design decisions (and why).
2. A "Getting Started for Developers" guide: prerequisites, local setup,
   running tests, common scripts, and how to make a first change.
3. Documentation of the most important modules: each module's responsibility,
   its public interface, and how it's used.
4. A glossary of domain-specific terms.
5. A simple diagram described in text or Mermaid syntax for the architecture.

Write clearly for a developer new to the project. Mark anything you need me
to fill in. Prioritize the docs that reduce onboarding time the most.
```

EXAMPLE RESULT

A documentation set — an `ARCHITECTURE.md`, a developer getting-started guide, key-module docs, a glossary, and a Mermaid diagram — written to get a new developer productive fast.

84

PRODUCTIVITY & CAREER

Write Code Comments and JSDoc

WHAT IS THIS PROMPT FOR?

Use this to add high-value comments and JSDoc/TSDoc to code — explaining the "why" behind tricky logic and documenting function signatures for editor tooltips and autocompletion.

AI PROMPT — copy & paste

PROMPT

```
You are an expert at writing useful code comments. Add high-quality comments
and JSDoc/TSDoc to the following code.

Code: [PASTE]
Language: [JS / TS]

Follow these rules:
1. Add JSDoc/TSDoc to exported functions/classes: a one-line summary, @param
   with types/meaning, @returns, @throws where relevant, and an @example for
   non-obvious usage.
2. Add INLINE comments only where they add value — explain WHY (intent,
   trade-offs, non-obvious decisions, gotchas, references to specs/issues),
   NOT what the code obviously does.
3. Do NOT add noise like "// increment i" or restate the code.
4. Flag any code that's so unclear it needs a comment as a sign it should be
   refactored instead — suggest the refactor.
5. Keep comments concise and maintainable (comments that won't quickly go
   stale).

Return the commented code, and a short note on anything you'd refactor rather
than comment.
```

EXAMPLE RESULT

The code annotated with proper JSDoc/TSDoc on exports and judicious "why" inline comments — no noise — plus suggestions where unclear code should be refactored rather than commented.

85

PRODUCTIVITY & CAREER

Write a Dockerfile and docker-compose Setup

WHAT IS THIS PROMPT FOR?

Use this when you need to containerize an app for consistent environments and deployment. Covers a production-grade, optimized Dockerfile and local multi-service orchestration.

AI PROMPT — copy & paste

PROMPT

```
Act as a DevOps engineer. Containerize my application with a production-grade
Dockerfile and a docker-compose setup for local development.

App: [E.G. "Node.js + Express + TypeScript API"], also needs: [E.G.
PostgreSQL, Redis]
Build/run commands: [E.G. npm run build, npm start]

Deliver:
1. A multi-stage Dockerfile that: installs deps efficiently (leveraging layer
   caching), builds the app, and produces a small final image (slim/alpine
   base, only production deps, non-root user).
2. A .dockerignore to keep the image clean and builds fast.
3. A docker-compose.yml for local dev wiring the app + database + cache, with
   environment variables, volumes for persistence, health checks, and
   sensible service dependencies (depends_on with conditions).
4. How to handle env vars/secrets (not baked into the image).
5. Commands to build, run, and tear down, plus how to view logs.

Explain each significant choice (multi-stage, non-root, caching) briefly.
```

EXAMPLE RESULT

A multi-stage, optimized, non-root Dockerfile, a `.dockerignore`, a `docker-compose.yml` wiring app/db/cache with health checks and volumes, secret-handling guidance, and the commands to run it all.

86

PRODUCTIVITY & CAREER

Set Up a CI/CD Pipeline with GitHub Actions

WHAT IS THIS PROMPT FOR?

Use this to automate testing, linting, building, and deployment on every push. A solid CI/CD pipeline catches bugs early and removes manual deployment steps.

AI PROMPT — copy & paste

PROMPT

```
You are a DevOps engineer. Create a CI/CD pipeline using GitHub Actions for
my project.

Project: [STACK — e.g. "Next.js app" / "Node.js API"], package manager: [npm/
pnpm/yarn], deploy target: [E.G. VERCEL / A VPS VIA SSH / DOCKER REGISTRY]

Create workflow file(s) that:
1. CI (on pull requests and pushes to main): install deps with caching, run
   lint, type-check, and tests; fail fast and run jobs in parallel where
   sensible.
2. Build the app and (if applicable) build/push a Docker image.
3. CD (on push/merge to main, after CI passes): deploy to the target, using
   GitHub Secrets for credentials — never hardcode secrets.
4. Use a matrix for multiple Node versions if useful, and concurrency control
   to cancel superseded runs.
5. Add a status badge snippet for the README.

Provide the complete .github/workflows/*.yml file(s) with comments, and list
exactly which GitHub Secrets I need to configure.
```

EXAMPLE RESULT

Complete GitHub Actions workflow file(s) with cached installs, parallel lint/type-check/test jobs, build and deploy steps using secrets, concurrency control, a README badge, and the exact secrets to set.

87

PRODUCTIVITY & CAREER

Write a Technical Blog Post

WHAT IS THIS PROMPT FOR?

Use this to turn something you built or learned into a clear technical article — great for personal branding, sharing knowledge, and demonstrating expertise to employers.

AI PROMPT — copy & paste

PROMPT

```
Act as a technical writer who creates clear, engaging developer content. Help
me write a technical blog post.

Topic: [WHAT YOU WANT TO WRITE ABOUT — e.g. "how I built X" / "explaining
concept Y"]
Audience level: [BEGINNER / INTERMEDIATE / ADVANCED]
Key points I want to cover: [LIST]
Goal: [TEACH / SHOWCASE A PROJECT / SHARE A LESSON]

Write a post that:
1. Opens with a hook and clearly states what the reader will learn/gain.
2. Is well-structured with descriptive headings and a logical progression.
3. Explains concepts clearly with analogies where helpful, building from
   simple to complex.
4. Includes well-commented, correct code examples that actually run.
5. Anticipates and answers the reader's likely questions and common pitfalls.
6. Ends with a concise summary and a clear next step / call to action.
7. Has an SEO-friendly title (give 3 options) and a meta description.

Keep the tone knowledgeable but approachable; avoid filler. Mark where I
should add screenshots or my own anecdotes.
```

EXAMPLE RESULT

A structured, engaging technical post with a hook, clear progression, runnable code examples, pitfall coverage, a summary and CTA, three SEO title options, and a meta description — with markers for your own additions.

88

PRODUCTIVITY & CAREER

Prepare for a Technical Interview

WHAT IS THIS PROMPT FOR?

Use this to prepare for coding and system-design interviews with targeted practice, explanations, and mock questions tailored to the role you're targeting.

AI PROMPT — copy & paste

PROMPT

```
Act as a technical interview coach who has interviewed at top tech companies.
Help me prepare for an interview.

Role: [E.G. "frontend developer" / "full-stack" / "backend"], level: [JUNIOR/
MID/SENIOR], company type: [STARTUP / BIG TECH / AGENCY]
My current strengths & weak areas: [BE HONEST]

Help me by:
1. Listing the topics likely to be tested for this role/level (DSA, system
   design, framework-specific, behavioral) and how heavily each is weighted.
2. Creating a focused prep plan for the next [X DAYS/WEEKS] prioritizing my
   weak areas.
3. Giving me 5 realistic technical questions for this role — but DON'T give
   answers yet; let me attempt them, then you review my solutions for
   correctness, edge cases, complexity, and communication.
4. For coding problems, coach me on explaining my thought process out loud
   (what interviewers actually score).
5. Providing 5 behavioral questions and the STAR-method structure to answer
   them.

Start with the topic breakdown and prep plan, then ask me which area to drill
first.
```

EXAMPLE RESULT

A weighted topic breakdown, a focused prep plan targeting your weak areas, realistic role-specific questions (held for you to attempt and get coached on), thought-process guidance, and behavioral prep — then a prompt to pick a starting area.

89

PRODUCTIVITY & CAREER

Optimize Your Developer Resume

WHAT IS THIS PROMPT FOR?

Use this to transform a flat resume into one that passes applicant tracking systems and impresses hiring managers — with quantified, impact-focused bullet points.

AI PROMPT — copy & paste

PROMPT

```
Act as a tech recruiter and resume expert who has screened thousands of
developer resumes. Review and improve mine.

Target role: [JOB TITLE], target companies: [TYPE], years of experience: [X]
My current resume content: [PASTE — experience bullets, skills, projects]
(Optional) Job description I'm targeting: [PASTE]

Improve it by:
1. Rewriting experience bullets to be impact- and result-focused using strong
   action verbs and QUANTIFIED outcomes (e.g. "reduced load time 40%",
   "served 10k users") — not just listed responsibilities. Where I lack
   numbers, ask me what to measure.
2. Tailoring keywords to the target role/JD so it passes ATS, without keyword
   stuffing.
3. Tightening to be concise and scannable (most-relevant first); flagging
   filler, clichés ("team player"), and anything to cut.
4. Advising on structure/section order and length for my experience level.
5. Strengthening the skills and projects sections (relevance over volume).

Give me the rewritten bullets and a prioritized list of changes. Ask me for
any missing metrics rather than inventing them.
```

EXAMPLE RESULT

Rewritten, quantified, impact-focused experience bullets, ATS-tuned keywords, tightened scannable structure, and a prioritized change list — with the AI asking for real metrics rather than fabricating them.

90

PRODUCTIVITY & CAREER

Write a Test Suite

WHAT IS THIS PROMPT FOR?

Use this when code lacks tests and you want meaningful unit and integration coverage — not just trivial tests, but ones that catch real bugs and document behavior.

AI PROMPT — copy & paste

PROMPT

```
You are a testing expert who values meaningful coverage over vanity metrics.
Write a thorough test suite for the following code.

Code under test: [PASTE]
Test framework: [JEST / VITEST / etc.], type: [UNIT / INTEGRATION]

Write tests that:
1. Cover the happy path AND the important edge cases: empty/null/undefined
   inputs, boundary values, invalid inputs, and error paths.
2. Follow Arrange-Act-Assert structure with clear, descriptive test names
   that document the expected behavior.
3. Test behavior and public contracts, not implementation details (so tests
   don't break on harmless refactors).
4. Mock external dependencies (DB, network, time, randomness) appropriately,
   and explain what to mock vs not.
5. Stay isolated and deterministic (no shared state, no order dependence).
6. Include at least one test that would FAIL if a likely bug were introduced.

Provide the full test file, and note any code that's hard to test and how to
refactor it for testability.
```

EXAMPLE RESULT

A meaningful test suite covering happy paths, edge cases, and error paths with clear names and AAA structure, appropriate mocking, deterministic isolation — plus notes on refactoring hard-to-test code.

91

PRODUCTIVITY & CAREER

Write an Architecture Decision Record (ADR)

WHAT IS THIS PROMPT FOR?

Use this when making a significant technical decision and you want to document the context, options, and reasoning so the team (and future-you) understands why — not just what.

AI PROMPT — copy & paste

PROMPT

```
Act as a staff engineer. Help me write an Architecture Decision Record (ADR)
for a technical decision.

The decision: [WHAT YOU'RE DECIDING — e.g. "choosing a state management
approach" / "adopting a message queue"]
Context: [THE PROBLEM, CONSTRAINTS, AND FORCES AT PLAY]
Options considered: [LIST THE OPTIONS YOU EVALUATED]
The decision I'm leaning toward: [IF ANY]

Write a clear ADR with these sections:
1. Title and status (Proposed / Accepted / Superseded).
2. Context: the problem and the forces/constraints driving the decision
   (technical and business).
3. Options considered: each with pros, cons, and trade-offs — fairly, not
   strawmanned.
4. Decision: what we chose and the concrete reasoning.
5. Consequences: the positive and negative results of this choice, and what
   becomes easier/harder.
6. Follow-ups / what would cause us to revisit this.

Keep it concise but complete. If my reasoning has gaps, point them out.
```

EXAMPLE RESULT

A complete, concise ADR — title/status, context with forces, fairly-evaluated options, the decision and reasoning, consequences, and revisit triggers — that captures why for the future, with gaps in your reasoning flagged.

92

PRODUCTIVITY & CAREER

Break Down a Project into Tasks

WHAT IS THIS PROMPT FOR?

Use this when facing a big, vague project and not knowing where to start. The AI decomposes it into ordered, manageable tasks you can actually begin.

AI PROMPT — copy & paste

PROMPT

```
Act as a senior engineer and technical project planner. Break down the
following project into a clear, ordered task plan I can execute.

Project: [DESCRIBE WHAT YOU WANT TO BUILD]
Stack/constraints: [...], timeline/effort available: [...], solo or team: [...]

Produce:
1. A short clarification of scope: the MVP (must-haves) vs nice-to-haves —
   call out anything that should be cut from v1.
2. A breakdown into phases/milestones, each delivering something usable.
3. Within each phase, concrete tasks ordered by DEPENDENCY (what must come
   first), each small enough to finish in a focused session, with a rough
   effort estimate (S/M/L).
4. Identify the riskiest/most-uncertain tasks to tackle early (de-risk first).
5. Note external dependencies, decisions to make upfront, and potential
   blockers.
6. A suggested "first 3 tasks to start today".

Present it as a structured, checklist-style plan. Bias toward shipping a thin
slice end-to-end before polishing.
```

EXAMPLE RESULT

A scoped MVP, a phased plan where each milestone is usable, dependency-ordered tasks with S/M/L estimates, early de-risking, noted blockers, and a concrete "start today" list — biased toward shipping a thin slice first.

93

PRODUCTIVITY & CAREER

Create a Study Plan for a New Technology

WHAT IS THIS PROMPT FOR?

Use this when you need to learn a new framework, language, or tool efficiently. The AI builds a structured, project-based learning path instead of aimless tutorial-hopping.

AI PROMPT — copy & paste

PROMPT

```
Act as an experienced engineer and mentor. Create a structured, project-based
study plan to help me learn [TECHNOLOGY — e.g. "Next.js" / "TypeScript" /
"Docker"].

My background: [WHAT I ALREADY KNOW], goal: [WHAT I WANT TO BUILD/ACHIEVE],
time available: [E.G. "1 hr/day for 4 weeks"].

Build a plan that:
1. Lists the core concepts I must learn, in dependency order (foundations
   before advanced), and explicitly excludes things I can skip for now.
2. Is project-based: for each phase, a small project that forces me to apply
   what I learned (learning by building, not just reading).
3. Fits my available time, broken into weekly goals and daily-sized chunks.
4. Recommends high-quality, specific resources (official docs first, plus
   reputable tutorials/courses) — avoid low-quality fluff.
5. Includes checkpoints to test my understanding (what I should be able to
   build/explain at each stage).
6. Warns about common beginner mistakes and misconceptions for this tech.

End with the capstone project that proves I've learned it.
```

EXAMPLE RESULT

A dependency-ordered, project-based study plan fitted to your time — weekly goals, applied mini-projects, vetted resources, understanding checkpoints, common-mistake warnings, and a capstone project.

94

PRODUCTIVITY & CAREER

Write API Documentation (OpenAPI)

WHAT IS THIS PROMPT FOR?

Use this when your API needs proper documentation for frontend devs, partners, or your future self. Produces clear reference docs and an OpenAPI spec.

AI PROMPT — copy & paste

PROMPT

```
You are a technical writer specializing in API documentation. Document my
REST API clearly and produce an OpenAPI 3 spec.

API details: [DESCRIBE OR PASTE ENDPOINTS — methods, paths, request/response
shapes, auth]

Deliver:
1. A human-readable overview: what the API does, the base URL, authentication
   (how to get and pass a token), and the standard error format.
2. For each endpoint: purpose, method + path, path/query params (with types
   and required flags), request body schema with an example, response schema
   with example(s), and the possible status codes/errors.
3. An OpenAPI 3.0 (YAML) specification covering the endpoints, schemas
   (components), and security scheme — valid and importable into Swagger UI/
   Postman.
4. Notes on pagination, rate limits, and versioning if applicable.
5. A "Getting Started" section with a copy-paste example request (curl).

Be precise and consistent. Mark anything you need me to confirm.
```

EXAMPLE RESULT

Human-readable API docs (overview, auth, per-endpoint reference with examples and status codes) plus a valid, importable OpenAPI 3 YAML spec, pagination/rate-limit notes, and a curl quickstart.

95

PRODUCTIVITY & CAREER

Estimate Project Time Realistically

WHAT IS THIS PROMPT FOR?

Use this when a client or manager asks "how long will this take?" and you want a defensible estimate that accounts for the things developers usually forget.

AI PROMPT — copy & paste

PROMPT

```
Act as an experienced engineering lead who estimates projects realistically.
Help me produce a defensible time estimate.

Project/feature: [DESCRIBE], my experience with this kind of work: [...],
known unknowns/risks: [...], working solo or team: [...]

Help me estimate by:
1. Breaking the work into tasks small enough to estimate individually (large
   tasks hide uncertainty).
2. Estimating each task with a range (optimistic / likely / pessimistic)
   rather than a single number.
3. Explicitly adding the things developers under-estimate: testing, code
   review, bug-fixing, edge cases, integration, deployment/devops, meetings/
   context-switching, and unknowns.
4. Applying a buffer for uncertainty proportional to how novel the work is,
   and explaining the reasoning (not a magic multiplier).
5. Separating "ideal focused hours" from "calendar time" (since real days
   aren't 8 focused hours).
6. Flagging the highest-uncertainty items that could blow the estimate, and
   suggesting a spike to de-risk them.

Give me a total range with the assumptions stated clearly.
```

EXAMPLE RESULT

A task-level estimate with optimistic/likely/pessimistic ranges, explicit time for the commonly-forgotten work, a reasoned uncertainty buffer, ideal-hours-vs-calendar-time, flagged risks, and a stated-assumption total range.

96

PRODUCTIVITY & CAREER

Refactor Toward Clean Architecture

WHAT IS THIS PROMPT FOR?

Use this when business logic is tangled with framework and database code, making the app hard to test and change. The AI restructures it toward clean, layered, maintainable architecture.

AI PROMPT — copy & paste

PROMPT

```
Act as a software architect who applies Clean Architecture pragmatically.
Help me restructure my code so business logic is decoupled from frameworks,
the database, and external services.

Current code/structure: [PASTE OR DESCRIBE — note where logic, DB access,
and HTTP handling currently live]
Stack: [E.G. Node.js + Express + Prisma]

Guide me to:
1. Identify the layers: entities/domain, use cases/services (business logic),
   interface adapters (controllers, presenters), and frameworks/drivers
   (Express, Prisma, external APIs).
2. Apply the Dependency Rule: dependencies point inward; the domain knows
   nothing about Express or Prisma. Use interfaces/ports for data access
   (repository pattern) so the DB is swappable and the logic is testable.
3. Show the target folder structure and where each responsibility goes.
4. Refactor a concrete example (one feature/use case) from my code into this
   structure, including a repository interface and its Prisma implementation.
5. Be pragmatic — call out where strict Clean Architecture would be
   over-engineering for my project size and what to simplify.

Show before → after for the example feature and explain the testability win.
```

EXAMPLE RESULT

A layered restructuring with the Dependency Rule applied, a repository-pattern data abstraction, a target folder structure, a concrete before→after refactor of one feature, and pragmatic guidance on not over-engineering.

97

PRODUCTIVITY & CAREER

Write a Tailored Job Application

WHAT IS THIS PROMPT FOR?

Use this when applying for a developer role and you want a cover letter or application message that's specific and compelling — not a generic template that gets ignored.

AI PROMPT — copy & paste

PROMPT

```
Act as a career coach for developers. Write a tailored, compelling cover
letter / application message for a job I'm applying to.

The job description: [PASTE]
My background: [RELEVANT EXPERIENCE, SKILLS, AND 1-2 PROJECTS/ACHIEVEMENTS]
Why I'm interested in THIS company/role: [GENUINE REASON, IF ANY]

Write a letter that:
1. Opens with a specific, genuine hook — why this role/company, not a generic
   "I am writing to apply".
2. Connects MY concrete experience and achievements directly to the key
   requirements in the job description (quantify where possible).
3. Demonstrates I understand what they do and the value I'd add, without
   flattery or fluff.
4. Stays concise (3-4 short paragraphs), confident but not arrogant, and in a
   natural human voice — not robotic or clichéd.
5. Ends with a clear, low-pressure call to action.

Avoid generic filler and overused phrases. If I'm missing a key requirement,
suggest how to honestly address it. Give me the letter plus 2 subject-line
options if it's an email.
```

EXAMPLE RESULT

A concise, tailored cover letter with a genuine hook, your achievements mapped to the job's requirements, demonstrated company understanding, a natural voice, a clear CTA, and email subject-line options — free of clichés.

98

PRODUCTIVITY & CAREER

Build Your Developer Personal Brand

WHAT IS THIS PROMPT FOR?

Use this when you want to grow your visibility — on LinkedIn, GitHub, or X — to attract opportunities. The AI helps you craft a strategy and content that reflect your real strengths.

AI PROMPT — copy & paste

PROMPT

```
Act as a personal-branding strategist for developers. Help me build an
authentic developer brand that attracts opportunities (jobs, clients,
network).

About me: [SKILLS, EXPERIENCE, NICHE/INTERESTS, WHAT I WANT — e.g. "land a
senior frontend role" / "get freelance clients"]
Platforms I'll use: [LINKEDIN / X / GITHUB / BLOG]

Help me with:
1. Positioning: a clear, honest one-line "what I do and who I help" statement,
   and the niche/themes I should be known for (based on my real strengths).
2. Profile optimization: specific improvements to my LinkedIn/GitHub/X bios
   and headlines so they're clear and compelling.
3. A content strategy: 3-5 content pillars I can post about consistently, and
   10 concrete post/topic ideas drawn from my actual work and learning.
4. A realistic, sustainable posting cadence given a busy schedule.
5. How to engage/network genuinely (not spammy) to grow reach.
6. What to AVOID (cringe, fake-guru tactics, dishonesty).

Keep it authentic — built on real skills and value, not hype.
```

EXAMPLE RESULT

A personal-brand plan: honest positioning and niche, concrete profile/bio improvements, content pillars with ten real post ideas, a sustainable cadence, genuine networking tactics, and a list of what to avoid.

99

PRODUCTIVITY & CAREER

Do a Self Code-Review Before Opening a PR

WHAT IS THIS PROMPT FOR?

Use this right before you push, to catch issues yourself — so your PRs get approved faster and you build a reputation for clean work. The review you do before others do.

AI PROMPT — copy & paste

PROMPT

```
Act as the strict but fair reviewer I want to impress. Review my changes as a
PRE-PR self-review so I can fix problems before anyone else sees them.

What this change does: [GOAL], the diff: [PASTE DIFF]

Run through a thorough self-review checklist and flag anything I should fix
first:
1. Does the code actually do what the goal says, and ONLY that (no
   accidental scope creep, no leftover debug logs / commented-out code / TODO
   I forgot)?
2. Correctness & edge cases: nulls, empties, error handling, async issues.
3. Tests: are the changes covered? What test is missing?
4. Readability: naming, dead code, magic values, anything a reviewer would
   nitpick — fix it now.
5. Security & performance red flags.
6. Did I update related docs/types/comments and not break anything adjacent?
7. Is the PR a reasonable SIZE, or should it be split for easier review?

Give me a prioritized "fix before pushing" list and a clean PR description +
title I can use.
```

EXAMPLE RESULT

A thorough pre-PR self-review: leftover-debug and scope checks, correctness/edge-case and security/performance flags, test-coverage gaps, readability nitpicks, a PR-size judgment, a prioritized fix list, and a ready-to-use PR title and description.

100

PRODUCTIVITY & CAREER

Create a Deployment Checklist

WHAT IS THIS PROMPT FOR?

Use this before deploying to production — especially your first deploy or a high-stakes release. A checklist prevents the small omissions that cause outages and 2 a.m. pages.

AI PROMPT — copy & paste

PROMPT

```
Act as a senior DevOps/SRE engineer. Create a thorough, practical
pre-deployment and deployment checklist for my application.

App & stack: [E.G. "Next.js frontend + Node.js API + PostgreSQL"], hosting:
[E.G. VERCEL / VPS / DOCKER / CLOUD], deploy method: [CI/CD / MANUAL]

Produce a checklist grouped into phases:
1. PRE-DEPLOY: tests/lint/build pass in CI, env vars/secrets set for prod,
   database migrations prepared and tested, dependencies audited, feature
   flags configured, and a rollback plan ready.
2. SECURITY & CONFIG: HTTPS/TLS, security headers, CORS, secrets not exposed,
   correct prod environment settings, rate limiting on.
3. PERFORMANCE & RELIABILITY: caching/CDN, asset optimization, health-check
   endpoint, error tracking + logging + monitoring/alerts configured.
4. DEPLOY: the deploy steps in order, how to run migrations safely (and in
   what order relative to the deploy), and how to verify with a smoke test.
5. POST-DEPLOY: verify key user flows, watch error rates/metrics, and the
   exact rollback procedure if something breaks.

Make it concrete and checkbox-style for my specific stack. Highlight the
top mistakes that cause production incidents.
```

EXAMPLE RESULT

A concrete, phase-grouped, checkbox-style deployment checklist tailored to your stack — pre-deploy, security/config, performance/reliability, the ordered deploy with safe migrations and smoke tests, and post-deploy verification with a rollback procedure.

BONUS

Developer Resources

You made it through all 100 prompts — here's your bonus. This is a hand-picked toolkit of the platforms, extensions, websites, and repositories that working developers actually use every day. Bookmark this section and return to it whenever you want to level up your stack.

100 hand-picked tools

- 25 Best AI Websites
- 25 Best VS Code Extensions
- 25 Best Developer Websites
- 25 Best GitHub Repositories

25 Best AI Websites

01 **ChatGPT (chat.openai.com)**
The most widely used AI assistant for coding, debugging, explanations, and rapid prototyping.

02 **Claude (claude.ai)**
An AI assistant known for long-context reasoning, careful code review, and high-quality writing.

03 **Google Gemini (gemini.google.com)**
Google's multimodal AI assistant with strong reasoning and tight Google ecosystem integration.

04 **GitHub Copilot (github.com/features/copilot)**
An AI pair programmer that autocompletes code directly inside your editor.

05 **Cursor (cursor.com)**
An AI-first code editor that lets you edit and generate code across your whole codebase via chat.

06 **Perplexity (perplexity.ai)**
An AI answer engine that searches the web and cites sources, great for technical research.

07 **v0 by Vercel (v0.dev)**
Generates production-ready React and Tailwind UI components from text prompts.

08 **Bolt.new (bolt.new)**
Builds and deploys full-stack web apps in the browser from a single prompt.

09 **Replit AI (replit.com)**
A cloud IDE with a built-in AI agent that writes, runs, and deploys code.

10 **Codeium / Windsurf (codeium.com)**
A free AI autocomplete and chat assistant supporting dozens of languages and editors.

11 **Tabnine (tabnine.com)**
A privacy-focused AI code completion tool that can run on your own infrastructure.

12 **Phind (phind.com)**
A developer-focused AI search engine optimized for programming questions.

13 **Hugging Face (huggingface.co)**
The hub for open-source AI models, datasets, and demos you can use in your projects.

14 **Poe (poe.com)**
A single interface to chat with many AI models (GPT, Claude, Gemini, and more) side by side.

15 **Ollama (ollama.com)**
Run powerful open-source LLMs locally on your own machine with one command.

16 **Mistral / Le Chat (chat.mistral.ai)**
A fast, capable AI assistant backed by strong open-weight models.

17 **DeepSeek (deepseek.com)**
A cost-effective AI model family with strong coding and reasoning performance.

18 **Warp (warp.dev)**
A modern terminal with built-in AI that turns natural language into shell commands.

19 **Codium / Qodo (qodo.ai)**
Generates meaningful tests and analyzes code quality automatically.

20 **Excalidraw + AI (excalidraw.com)**
A whiteboard tool that turns text prompts into diagrams and wireframes.

21 **Whimsical AI (whimsical.com)**
Generates flowcharts, mind maps, and diagrams from natural language.

22 **Napkin AI (napkin.ai)**
Turns your text into clean visuals and diagrams for docs and presentations.

23 **Grok (grok.com)**
xAI's conversational assistant with real-time access to current information.

24 **Sourcegraph Cody (sourcegraph.com/cody)**
An AI assistant that understands and answers questions about large codebases.

25 **AskCodi (askcodi.com)**
An AI coding companion for generating code, tests, and documentation.

25 Best VS Code Extensions

01 **Prettier**
Automatically formats your code to a consistent style on save.

02 **ESLint**
Catches bugs and enforces code-quality rules in JavaScript and TypeScript as you type.

03 **GitLens**
Supercharges Git inside VS Code with blame annotations, history, and authorship insights.

04 **GitHub Copilot**
Provides AI-powered code completions and chat right in your editor.

05 **Live Server**
Launches a local development server with live reload for static and dynamic pages.

06 **Tailwind CSS IntelliSense**
Adds autocomplete, linting, and hover previews for Tailwind classes.

07 **Path Intellisense**
Autocompletes filenames and import paths as you type.

08 **Auto Rename Tag**
Automatically renames the matching opening/closing HTML or JSX tag.

09 **Error Lens**
Highlights errors and warnings inline directly on the offending line.

10 **Thunder Client**
A lightweight REST API client built into VS Code, like a mini Postman.

11 **REST Client**
Sends HTTP requests from a plain `.http` file and views responses inline.

12 **Material Icon Theme**
Gives your files and folders clear, recognizable icons.

13 **Code Spell Checker**
Catches spelling mistakes in code, comments, and strings.

14 **Import Cost**
Shows the size of imported packages inline so you can spot heavy dependencies.

15 **DotENV**
Adds syntax highlighting for `.env` environment variable files.

16 **Docker**
Manages containers, images, and Dockerfiles directly from the editor.

17 **Prisma**
Provides syntax highlighting, formatting, and autocomplete for Prisma schema files.

18 **ES7+ React/Redux Snippets**
Speeds up React development with shorthand snippets for components and hooks.

19 **Better Comments**
Color-codes comments by type (TODO, alerts, questions) for clarity.

20 **Peacock**
Color-codes your VS Code workspace so you can tell projects apart at a glance.

21 **Todo Tree**
Collects all your TODO and FIXME comments into a searchable tree view.

22 **Live Share**
Enables real-time collaborative editing and debugging with teammates.

23 **CodeSnap**
Generates beautiful screenshots of your code for sharing and docs.

24 **Markdown All in One**
Adds shortcuts, table of contents, and live preview for Markdown files.

25 **IntelliCode**
Provides AI-assisted, context-aware autocompletion based on best practices.

25 Best Websites Every Developer Uses

01 **MDN Web Docs (developer.mozilla.org)**
The definitive reference for HTML, CSS, JavaScript, and web APIs.

02 **Stack Overflow (stackoverflow.com)**
The largest Q&A community for solving specific programming problems.

03 **GitHub (github.com)**
The home of open-source code, version control, and developer collaboration.

04 **freeCodeCamp (freecodecamp.org)**
Free, comprehensive coding curricula and thousands of tutorials.

05 **W3Schools (w3schools.com)**
Beginner-friendly references and interactive examples for web technologies.

06 **DevDocs (devdocs.io)**
Fast, searchable, offline-capable documentation for hundreds of technologies in one place.

07 **CodePen (codepen.io)**
An online playground for building and sharing front-end snippets and demos.

08 **CanIUse (caniuse.com)**
Shows browser support tables for modern web features.

09 **Roadmap.sh (roadmap.sh)**
Visual learning roadmaps for front-end, back-end, DevOps, and more.

10 **LeetCode (leetcode.com)**
Coding challenges and interview prep for data structures and algorithms.

11 **HackerRank (hackerrank.com)**
Practice problems and skill assessments used by many employers.

12 **Dev.to (dev.to)**
A friendly community blogging platform for developer articles and discussion.

13 **Hacker News (news.ycombinator.com)**
Tech and startup news curated by the developer community.

14 **CSS-Tricks (css-tricks.com)**
Deep, practical articles and guides on CSS and front-end techniques.

15 **Smashing Magazine (smashingmagazine.com)**
In-depth articles on web design, UX, and development.

16 **Regex101 (regex101.com)**
Build, test, and debug regular expressions with live explanations.

17 **JSON Formatter (jsonformatter.org)**
Formats, validates, and beautifies JSON quickly.

18 **Excalidraw (excalidraw.com)**
A virtual whiteboard for sketching architecture and flow diagrams.

19 **Carbon (carbon.now.sh)**
Creates beautiful, shareable images of your source code.

20 **Postman (postman.com)**
The industry-standard tool for building, testing, and documenting APIs.

21 **Vercel (vercel.com)**
Deploy front-end and full-stack apps with zero-config and instant previews.

22 **Netlify (netlify.com)**
Build, deploy, and host modern web projects with continuous deployment.

23 **Coolors (coolors.co)**
Generate, tweak, and export color palettes in seconds.

24 **Unsplash (unsplash.com)**
A huge library of free, high-quality images for your projects.

25 **Stack Blitz (stackblitz.com)**
A full in-browser IDE for instantly spinning up web projects.

25 Best GitHub Repositories

01 **freeCodeCamp/freeCodeCamp**
The open-source codebase and curriculum behind the freeCodeCamp learning platform.

02 **EbookFoundation/free-programming-books**
A massive curated list of free programming books in many languages.

03 **sindresorhus/awesome**
The original "awesome" list linking to curated lists on nearly every tech topic.

04 **kamranahmedse/developer-roadmap**
Community-maintained roadmaps for becoming different kinds of developer.

05 **trekhleb/javascript-algorithms**
Algorithms and data structures implemented and explained in JavaScript.

06 **jwasham/coding-interview-university**
A complete, self-paced study plan for landing a software engineering job.

07 **donnemartin/system-design-primer**
A widely used guide to learning how to design large-scale systems.

08 **public-apis/public-apis**
A categorized directory of free public APIs for building projects.

09 **practical-tutorials/project-based-learning**
A curated list of tutorials that teach by building real projects.

10 **30-seconds/30-seconds-of-code**
Short, reusable code snippets for everyday programming tasks.

11 **goldbergyoni/nodebestpractices**
A comprehensive collection of Node.js best practices.

12 **airbnb/javascript**
Airbnb's influential and widely adopted JavaScript style guide.

13 **type-challenges/type-challenges**
Exercises to master TypeScript's type system.

14 **vercel/next.js**
The source and examples for the Next.js React framework.

15 **facebook/react**
The official React library repository, including issues and releases.

16 **tailwindlabs/tailwindcss**
The source for the Tailwind CSS utility-first framework.

17 **prisma/prisma**
The next-generation ORM for Node.js and TypeScript.

18 **microsoft/TypeScript**
The official repository for the TypeScript language and compiler.

19 **getify/You-Dont-Know-JS**
A deep book series on the core mechanics of JavaScript.

20 **florinpop17/app-ideas**
A collection of application ideas to improve your coding skills by building.

21 **bradtraversy/design-resources-for-developers**
A curated list of design and UI resources for developers.

22 **denoland/deno**
A modern, secure runtime for JavaScript and TypeScript.

23 **ryanmcdermott/clean-code-javascript**
Clean Code principles adapted for JavaScript with examples.

24 **github/gitignore**
A collection of useful `.gitignore` templates for every language and tool.

25 **abi/screenshot-to-code**
An open-source tool that turns a screenshot of a UI into working code.

FINAL WORDS

You’re ready to build.

Congratulations — you now hold a complete, practical toolkit of **100 professional AI prompts** plus a curated set of developer resources.

Here's the truth most people miss: the developers who get the most out of AI aren't the ones who type "write me some code." They're the ones who give clear context, constraints, and structure — exactly like the prompts in this book. Used well, AI becomes the fastest, most patient senior engineer you've ever worked with.

How to get the most value

- **Don't just copy — customize.** Replace every `[SQUARE BRACKET]` placeholder with your real details. The more specific your input, the better the output.
- **Iterate.** Treat the first AI response as a draft. Ask follow-ups, request changes, and push for better.
- **Verify everything.** AI is a brilliant assistant, not an infallible oracle. Read, test, and understand the code before you ship it.
- **Build your own library.** When a prompt works well for you, save your customized version. Over time you'll have a personal collection tuned to your exact workflow.

Keep going

Pick one prompt today and use it on something real you're working on. Then another tomorrow. Small, consistent use is what turns these pages into actual saved hours, shipped features, and a sharper skill set.

You have everything you need. Now go build something great.

SERVICES

Work With Web.Lib

From idea to launch — we build the digital products that move your business forward.

Website Development

Modern, responsive websites built for businesses, startups and personal brands.

Business Systems

Custom web applications, dashboards and internal management systems.

UI/UX Design

Modern interfaces focused on user experience, speed and conversion.

WEBSITE
web-lib-one.vercel.app

PHONE
+218 94 100 0916

SCAN TO START A PROJECT
web-lib-one.vercel.app

Web.Lib

Let’s Build Your Next Project

Need a modern website or custom business system?

- Business Websites
- Company Websites
- Dashboards
- Web Applications
- Admin Panels
- AI Integrations
- UI/UX Design

WEBSITE
web-lib-one.vercel.app

PHONE
+218 94 100 0916

SCAN TO EXPLORE OUR WORK

Let’s turn your idea into a professional digital product.

web-lib-one.vercel.app · instagram.com/web.libly · @web.lib